

PLAN TECNICO DEBACU EVALUATION

Tu análisis es correcto en lo esencial, y además detecta un problema de modelo, no solo de UI.

Ahora mismo tienes mezclados dos niveles distintos:

- **nivel org:** el cliente/hotel grupo que paga y agrupa propiedades
- **nivel propiedad:** el hotel concreto donde ocurre una incidencia, se sube un CSV o se calcula revenue

Si no separas eso bien, el sistema da datos “válidos” pero operativamente ambiguos. Y eso a medio plazo te rompe reporting, IA, auditoría y utilidad real.

Mi valoración, punto por punto

1. Consulta automática CSV: debe llevar selector de propiedad

Sí, sin discusión.

Porque un CSV de screening o de reservas no pertenece al “grupo” en abstracto. Pertenecce a una propiedad concreta, salvo que el fichero venga explícitamente consolidado y además lo soportes como caso especial.

Si hoy un usuario de una org con 2 hoteles sube un CSV sin indicar propiedad:

- no sabes a qué hotel imputar las incidencias/resultados
- mezclas riesgo de hoteles distintos
- luego no puedes explicar a la IA ni al reporting dónde está el problema real
- te cargas cualquier análisis por hotel, canal o segmento

Conclusión práctica:

La consulta automática CSV debe obligar a seleccionar:

- propiedad
- opcionalmente tipo de importación, si luego quieres soportar “propiedad” vs “grupo”

Pero por ahora, lo sensato es:

- **1 CSV = 1 propiedad**
-

2. Registrar incidencia manual: también debe llevar selector de propiedad

También correcto.

Una incidencia no ocurre “en la org”. Ocurre en:

- una propiedad concreta
- idealmente también en una reserva / estancia / huésped concreto si existe
- opcionalmente canal, segmento, fecha, habitación, etc.

Si no guardas propiedad en la incidencia, luego no puedes responder cosas básicas como:

- qué hotel del grupo tiene más riesgo
- qué canal falla en hotel4 y no en demohotel
- qué propiedad concentra pérdidas
- dónde hay más reincidencia

Aquí no es opcional.

La incidencia debe tener property_id.

Y el formulario debe:

- obligar a seleccionar propiedad
- o autoseleccionarla si el usuario entra ya “dentro” del contexto de una propiedad

Pero no dejarla en org a secas.

3. Consulta manual: también debería estar ligada a propiedad

Aquí haría un matiz.

Tu idea general es buena, pero hay dos casos distintos:

Caso A. Consulta manual operativa previa a una reserva o check-in

Esto sí debe poder asociarse a propiedad, porque la consulta la hace un hotel concreto del grupo.

Ejemplo:

- el huésped va a HOTEL_4
- la consulta la lanza HOTEL_4
- el riesgo detectado debe quedar imputado a HOTEL_4

Caso B. Consulta manual centralizada del grupo

Si una central o owner consulta a nivel corporativo, puede tener sentido permitir consulta a nivel org, pero aun así el sistema debería pedir:

- o propiedad destino
- o marcarla como “consulta corporativa / sin propiedad asignada”

Mi recomendación práctica:

- **hazla obligatoria por propiedad** en la UI estándar
- si luego quieres un modo corporativo, lo añades aparte

Porque si no, vuelves a mezclar cosas.

4. Revenue: las tres primeras opciones no pueden quedarse como están si el selector no filtra de verdad

Correcto.

Si tienen selector de propiedad pero internamente siguen mostrando datos por org, eso ahora mismo es engañoso. No es un fallo menor de UX; es un fallo funcional.

Porque el usuario cree que está viendo HOTEL_4, pero en realidad ve agregado del grupo.

Eso hay que arreglar ya de una de estas dos formas:

- o el selector filtra de verdad por `property_id`
- o quitas el selector hasta que funcione

Dejar un selector decorativo es mala idea.

Tu propuesta de mover:

- Análisis por Canal
- Nivel de Riesgo
- Fugas de Revenue

a un bloque tipo **“Riesgo por Propiedad”** o **“Revenue Risk por Propiedad”**

me parece bien.

De hecho, estructuralmente tiene más sentido así:

Propuesta de menú más coherente

Operativa / Riesgo

- Consulta automática CSV
- Consulta manual
- Registrar incidencia

Riesgo por Propiedad

- Análisis por canal
- Nivel de riesgo
- Fugas de revenue

Revenue

- Importación revenue
- Día x Día
- Mensual
- Canales & segmentos
- Pickup avanzado

Auditoría

- Estadísticas operativas

- Histórico
- Exportaciones

Así separas mejor:

- **captura del riesgo**
 - **análisis del riesgo por hotel**
 - **producción revenue**
 - **auditoría global**
-

Donde tu análisis es especialmente bueno

Has visto bien que **Auditoría sí puede vivir a nivel org.**

Eso es correcto.

5. Estadísticas operativas

Tiene sentido que sea org/global si su objetivo es responder:

- cuántas consultas se han hecho en el grupo
- cuántos registros se han añadido
- cuál es el riesgo agregado
- qué actividad tiene toda la cuenta

Eso sirve al owner o dirección del grupo.

Pero aquí hay una condición:

aunque el KPI principal sea org, debes poder desglosar por propiedad.

Es decir, la vista puede abrir en total grupo, pero luego debería tener:

- breakdown por propiedad
- ranking de propiedades con más riesgo
- distribución por canal/segmento y propiedad
- evolución por propiedad

Si no, la parte IA queda coja.

6. La IA en Estadísticas Operativas: aquí está el matiz importante

También aciertas: la IA metida “tal cual” en esa pantalla, sin contexto de propiedad, queda floja.

No porque esté mal conceptualmente, sino porque el insight útil no es:

- “tu org tiene riesgo medio X”

Eso es demasiado genérico.

El insight útil es más bien:

- qué propiedad del grupo concentra más riesgo
- qué canal genera más pérdida en esa propiedad
- qué segmento repite más incidencias
- dónde hay anomalías vs el resto del grupo
- qué propiedad debería revisar política comercial o screening

Por tanto:

La IA sí puede vivir en auditoría org, **pero no como dato plano**, sino como capa de interpretación cross-property.

Ejemplos de insights buenos:

- “HOTEL_4 concentra el 62% del net loss del grupo en los últimos 30 días.”
- “DEMOHOTEL presenta más incidencia por canal directo, mientras HOTEL_4 concentra riesgo en OTA/B2B.”
- “El grupo tiene riesgo medio estable, pero el aumento reciente procede casi exclusivamente de HOTEL_4.”
- “El segmento walk-in aporta poco volumen pero alto impacto relativo en DEMOHOTEL.”

Eso sí tiene valor.

7. Histórico de consultas

Sí, mantenerlo a nivel org tiene sentido.

Porque el histórico es trazabilidad:

- quién consultó
- cuándo
- qué hizo
- sobre qué entidad

Pero debes guardar siempre suficiente contexto para discriminar:

- org_id
- property_id nullable o no, según el caso
- tipo de acción
- usuario

- input / resultado / export

La vista puede ser org, pero con filtro por propiedad.

Eso es lo correcto.

8. Exportaciones

También correcto: pueden ser org, pero con discriminación por propiedad.

Lo razonable aquí es permitir:

- export global del grupo
- export por propiedad
- export comparativo entre propiedades

No lo limitaría a una sola opción.

En la UI:

- ámbito: GRUPO o PROPIEDAD
- si PROPIEDAD, selector de propiedad
- si GRUPO, opción de incluir columna propiedad o agrupar por propiedad

Así cubres ambos usos.

Conclusión clara

Sí, tu análisis es correcto.

Y la regla de fondo debería quedar así:

Regla buena de arquitectura

Todo lo operativo real debe llevar `property_id`

Eso incluye:

- incidencias
- consultas manuales
- consultas CSV
- screening results
- alertas asociadas a hotel
- riesgo por canal/segmento cuando procede
- revenue risk

Regla de reporting

Todo lo ejecutivo puede verse a nivel org, pero debe poder desglosarse por `property_id`

Eso incluye:

- estadísticas operativas
 - histórico
 - exportaciones
 - IA corporativa
 - resúmenes de grupo
-

Mi recomendación práctica, sin adornos

Haz esto en este orden:

Fase 1. Corregir captura

Primero arregla el origen del dato:

1. **Registrar incidencia** → selector obligatorio de propiedad
2. **Consulta automática CSV** → selector obligatorio de propiedad
3. **Consulta manual** → selector obligatorio de propiedad

Si no corriges esto primero, todo lo demás sale contaminado.

Fase 2. Corregir pantallas “falsamente por propiedad”

Después arregla:

4. **Análisis por Canal**
5. **Nivel de Riesgo**
6. **Fugas de Revenue**

para que filtren de verdad por `property_id`.

Fase 3. Reordenar menú

Luego ya haces el ajuste de navegación:

7. crear bloque **Riesgo por Propiedad**
8. mover ahí esas 3 pantallas

Fase 4. Refinar auditoría org

Después:

9. mantener Auditoría a nivel org
 10. añadir breakdown por propiedad
 11. meter IA corporativa comparativa entre propiedades
-

Donde te diría que tengas cuidado

Hay una trampa aquí:

Si cambias solo frontend y añades selectores, pero en BD o Edge Functions sigues guardando solo `org_id`, no has arreglado nada.

Necesitas revisar que en el backend también exista y se propague:

- `property_id` en tablas relevantes
- validación de que la propiedad pertenece a la org del usuario
- filtros por `property_id` en Edge Functions
- RLS o checks de seguridad coherentes

Si eso no está, el selector será cosmético.

Decisión

Debacu es multi-property dentro de una org.

Por tanto:

- la unidad operativa es la **propiedad**
- la unidad de gobierno/reporting es la **org**

Esa es la separación correcta.

Si quieres, en el siguiente mensaje te dejo esto convertido en un plan técnico exacto con:

- qué tablas deben llevar `property_id`
- qué pantallas cambian
- qué Edge Functions hay que tocar
- y en qué orden hacerlo sin romper lo que ya tienes.

Entonces lo cerramos con una regla simple y no ambigua:

org = ámbito de gobierno y reporting

property = ámbito operativo real

A partir de ahí, todo lo que capture hechos debe quedar atado a propiedad. Y todo lo ejecutivo puede agregarse por org, pero con posibilidad de desglosar por propiedad.

Modelo objetivo cerrado

1. Nivel org

Sirve para:

- cuenta, suscripción, seats, usuarios
- auditoría global
- histórico global
- exportaciones globales
- KPIs agregados del grupo
- IA comparativa entre propiedades del grupo

2. Nivel propiedad

Sirve para:

- incidencias
- consultas manuales
- consultas automáticas CSV
- screening results
- revenue risk
- análisis por canal
- nivel de riesgo
- fugas de revenue
- imports revenue
- producción diaria/mensual
- pickup

Esa separación es la correcta y ya no la tocaría.

Qué hay que cerrar técnicamente

A. Entidades que deben llevar `property_id`

Esto es lo primero que hay que revisar.

Obligatorio

Deben tener `property_id` real, no opcional decorativo:

- incidencias / evaluations
- consultas manuales
- screening runs
- screening results
- imports CSV de screening
- revenue alerts
- revenue daily / snapshots / imports
- cualquier tabla que represente un hecho ocurrido en un hotel concreto

Puede ser org con desglose por propiedad

Aquí puede vivir a nivel org, pero guardando referencia a propiedad cuando exista:

- audit history
 - export jobs / export logs
 - estadísticas operativas agregadas
 - eventos de uso de IA
 - histórico de descargas
-

B. Regla backend

Toda Edge Function que cree o lea datos operativos debe aceptar y validar:

- `org_id`
- `property_id`

Y además comprobar que:

- esa propiedad pertenece a esa org
- el usuario autenticado pertenece a esa org
- si aplica, tiene acceso a esa propiedad

Esto es importante porque si no el selector existe pero la lógica sigue rota.

Cambios funcionales por pantalla

1. Consulta automática CSV

Estado deseado

Debe obligar a elegir propiedad antes de subir el CSV.

Flujo correcto

- selector de propiedad arriba
- el usuario elige HOTEL_4 o DEMOHOTEL
- sube CSV
- el import queda ligado a esa propiedad
- screening e incidencias derivadas quedan imputadas a esa propiedad

Decisión

Por ahora:

1 CSV = 1 propiedad

No intentaría soportar CSV multi-property todavía. Solo complica todo.

2. Consulta manual

Estado deseado

Debe tener selector de propiedad.

Regla

La consulta se imputa a la propiedad desde la que se hace o a la propiedad destino del huésped.

Resultado

El historial, scoring y alertas deben quedar ligados a esa propiedad.

3. Registrar incidencia

Estado deseado

Debe tener selector obligatorio de propiedad.

Mejor aún

Si puedes, además:

- propiedad
- canal
- segmento
- reserva opcional
- huésped opcional
- fecha del hecho

Pero lo mínimo imprescindible es propiedad.

Sin eso, la incidencia queda mal modelada.

4. Revenue risk por propiedad

Las tres pantallas:

- Análisis por Canal
- Nivel de Riesgo
- Fugas de Revenue

deben pasar a ser **100% property-based**.

Ahora mismo

Según lo que has visto, el selector está pero no filtra de verdad.

Eso hay que corregir. Si no, el dato engaña.

Estado deseado

- selector de propiedad funcional
 - queries filtradas por `property_id`
 - insights para esa propiedad
 - posibilidad futura de comparar con media org
-

5. Auditoría

Aquí sí mantendría enfoque org.

Estadísticas operativas

Vista principal:

- total consultas grupo
- total registros grupo

- riesgo agregado grupo

Pero con bloque adicional:

- desglose por propiedad
- top propiedades con más riesgo
- top canales/segmentos de riesgo del grupo

IA

No como “alerta por propiedad individual” metida con calzador ahí.

Sí como interpretación del grupo:

- qué propiedad concentra más riesgo
- qué canal/segmento concentra más pérdida
- qué hotel se desvía del resto

Ahí sí encaja.

6. Histórico

Mantenerlo por org es correcto.

Pero debe permitir filtrar por:

- propiedad
- tipo de acción
- usuario
- fecha

Y cada fila debería llevar contexto:

- org
 - property
 - acción
 - actor
 - timestamp
-

7. Exportaciones

También correcto a nivel org, pero con ámbito configurable.

Debe permitir

- export de grupo
 - export por propiedad
 - export grupo con columna propiedad
 - export comparativo por propiedad
-

Reorganización de menú

Yo dejaría algo así.

Operativa

- Dashboard
- Consulta automática (CSV)
- Consulta manual
- Registrar incidencia

Riesgo por Propiedad

- Análisis por Canal
- Nivel de Riesgo
- Fugas de Revenue

Revenue Intelligence

- Importación Revenue
- Día x Día
- Mensual
- Canales & Segmentos
- Pickup Avanzado

Auditoría

- Estadísticas operativas
- Histórico
- Exportaciones

Esto ya se entiende mejor y evita mezclar captura con análisis y auditoría.

Plan técnico por fases

Fase 1 — cerrar el modelo de datos

Objetivo: que todo hecho operativo tenga `property_id`.

Revisar tablas

Hay que clasificar tablas en tres grupos:

Grupo A. Deben llevar `property_id` obligatorio

Ejemplo:

- `debacu_evaluations` o equivalente
- tablas de screening/import/manual consult
- `revenue alerts`
- `revenue imports / daily / snapshots`

Grupo B. Deben llevar `property_id` nullable

Ejemplo:

- `audit history`
- `export logs`
- `action logs`

Grupo C. Org puro

Ejemplo:

- `org members`
- `subscriptions`
- `billing`
- `seats`
- `settings de org`

Resultado esperado

Tener una lista cerrada tabla por tabla y no seguir improvisando.

Fase 2 — backend / Edge Functions

Objetivo: que backend ya trabaje bien aunque el frontend aún no esté pulido.

Tocar primero

1. create manual consultation
2. create csv screening/import
3. create incident/evaluation
4. revenue risk endpoints de las 3 pantallas
5. audit stats si quieres desglose por propiedad

En todas

- entrada con `property_id`
 - validación propiedad $\in$ org
 - persistencia de `property_id`
 - filtros por `property_id` al leer
-

Fase 3 — frontend operativo

Objetivo: que el usuario ya no pueda generar datos ambiguos.

Cambios

1. Consulta automática CSV → selector obligatorio de propiedad
2. Consulta manual → selector obligatorio de propiedad
3. Registrar incidencia → selector obligatorio de propiedad

Esto es prioritario.

Fase 4 — frontend analítico

Objetivo: que las pantallas de riesgo por propiedad dejen de mentir.

Cambios

4. Análisis por Canal → filtrar por propiedad real
 5. Nivel de Riesgo → filtrar por propiedad real
 6. Fugas de Revenue → filtrar por propiedad real
-

Fase 5 — auditoría org bien resuelta

Objetivo: mantener visión global sin perder desglose.

Cambios

7. Estadísticas operativas → añadir breakdown por propiedad
 8. Histórico → filtro por propiedad
 9. Exportaciones → ámbito org / propiedad
-

Lo que no haría ahora

Para no dispersarte, yo no haría todavía:

- CSV multi-property
- permisos finos por propiedad por usuario, salvo que ya los necesites
- comparativas complejas entre propiedades en todas las pantallas
- IA generativa en todas partes

Primero cierra la base del dato. Luego el resto.

Orden real de trabajo recomendado

Sprint 1

- definir tablas afectadas
- decidir dónde `property_id` es obligatorio
- migraciones SQL

Sprint 2

- Edge Function consulta manual
- Edge Function CSV screening
- Edge Function registrar incidencia

Sprint 3

- añadir selector de propiedad en esas 3 pantallas
- guardar y leer ya con property real

Sprint 4

- corregir Análisis por Canal

- corregir Nivel de Riesgo
- corregir Fugas de Revenue

Sprint 5

- reorganizar menú
 - reforzar auditoría org con breakdown por propiedad
 - IA comparativa de grupo
-

Decisión funcional final

La dejo formulada de forma corta para que no haya dudas:

Regla 1

Toda acción operativa debe quedar asociada a una propiedad.

Regla 2

Toda vista ejecutiva puede agregarse por org, pero debe poder discriminar por propiedad.

Regla 3

No se mostrará ningún selector de propiedad si el backend no filtra realmente por propiedad.

Regla 4

No se permitirá crear incidencias, consultas o imports sin propiedad seleccionada.

Siguiente paso útil

El siguiente paso correcto ya no es debatir más el modelo. Es bajarlo a ejecución.

En el próximo mensaje te dejo, si quieres, el **plan técnico exacto tabla por tabla y función por función**, empezando por:

- qué tablas revisar primero
- qué campos añadir
- y qué pantallas tocar antes para no romper nada.

Sí. Y ese matiz cambia bastante el mapa. No son solo “4 opciones más”; en realidad son la **base maestra de Revenue por propiedad**. Si eso existe ya, entonces el modelo queda más claro todavía.

Lo que tienes en Revenue Intelligence se divide realmente en **dos capas**:

1. Capa maestra de configuración por propiedad

Estas cuatro pantallas no son reporting. Son **setup operativo**:

- **Propiedades**
- **Tipos de habitación**
- **Calendario de precios**
- **Eventos y temporadas**

Y todas dependen de `property_id` de forma natural.

Lectura correcta de esas cuatro

Propiedades

Es nivel **org** para gestión, porque ahí das de alta las propiedades del grupo.

Pero cada fila representa una propiedad concreta.

Es un maestro org → propiedades.

Tipos de habitación

Es claramente **property-based**.

Un tipo de habitación no pertenece al grupo abstracto; pertenece a una propiedad.

Calendario de precios

También **property-based**.

Además depende de:

- propiedad
- tipo de habitación
- reglas
- eventos/temporadas
- rango de fechas

Aquí no hay ninguna duda.

Eventos y temporadas

También **property-based** si afectan pricing, demanda y ocupación de una propiedad concreta.

Luego quizá puedas tener eventos “compartidos” a nivel grupo, pero el modelo operativo bueno es:

- evento/temporada aplicado a una propiedad
 - y si quieres replicarlo a varias, lo haces como acción de sistema, no como entidad ambigua
-

Lo que esto implica de verdad

Tu arquitectura queda mejor expresada así:

A. Revenue Intelligence tiene dos subbloques

Revenue Setup / Configuración por propiedad

- Propiedades
- Tipos de habitación
- Calendario de precios
- Eventos y temporadas

Revenue Analysis / Análisis por propiedad

- Producción Día x Día
- Mensual
- Canales & Segmentos
- Pickup Avanzado
- Análisis por Canal
- Nivel de Riesgo
- Fugas de Revenue
- Importación Revenue

Eso ya tiene coherencia.

Porque ahora mismo estabas mezclando en el mismo bloque:

- maestros/configuración
- ingestión
- reporting
- riesgo

Funciona visualmente, pero conceptualmente está mezclado.

Clasificación correcta de todo el módulo

1. Org-level puro

Estos viven a nivel grupo:

- suscripción

- usuarios / miembros
- seats
- auditoría global
- histórico global
- exportaciones globales
- alta y gestión del catálogo de propiedades del grupo

2. Property-level maestro

Estos son configuración base de cada hotel:

- tipos de habitación
- calendario de precios
- eventos y temporadas
- reglas de pricing si están ligadas a una propiedad
- imports ligados a una propiedad

3. Property-level analítico

Estos leen datos de una propiedad:

- importación revenue
- producción día x día
- mensual
- canales & segmentos
- pickup
- análisis por canal
- nivel de riesgo
- fugas de revenue

4. Org-level analítico agregado

Estos son cuadros de mando corporativos:

- estadísticas operativas
- histórico
- exportaciones
- IA resumen del grupo
- comparativas entre propiedades

Lo importante: ya no debes pensar “Revenue Intelligence” como un solo saco

Debes pensarlo así:

Revenue Intelligence

Configuración

- Propiedades
- Tipos de habitación
- Calendario de precios
- Eventos y temporadas

Operación / Ingesta

- Importación Revenue

Análisis por propiedad

- Día x Día
- Mensual
- Canales & Segmentos
- Pickup Avanzado
- Análisis por Canal
- Nivel de Riesgo
- Fugas de Revenue

Eso encaja mucho mejor con lo que ya has construido.

Tu conclusión anterior sigue siendo válida, pero ampliada

Antes habíamos dicho:

- org = gobierno
- property = operación

Sigue siendo correcto, pero ahora añadimos otra precisión:

Propiedades es un maestro de org

Aunque esté dentro de Revenue, **Propiedades** no es “análisis por propiedad”; es **administración del catálogo de propiedades del grupo**.

Por tanto:

Propiedades

- ámbito: org
- función: alta/configuración de propiedades del grupo

Todo lo demás que cuelga de Propiedades

- ámbito: property

Eso es exactamente lo que muestran tus capturas.

Dependencias reales entre pantallas

Y aquí está lo clave para no seguir montando cosas sueltas.

Cadena maestra correcta

1. Propiedad

Primero existe la propiedad.

2. Tipos de habitación

Luego defines los room types de esa propiedad.

3. Eventos y temporadas

Luego defines contexto temporal de esa propiedad.

4. Calendario de precios

Luego el pricing diario se nutre de:

- propiedad
- tipos de habitación
- reglas
- eventos/temporadas

5. Importación Revenue

Luego importas producción real de esa propiedad.

6. Reporting Revenue

Luego puedes sacar:

- día x día
- mensual
- pickup
- canales & segmentos

7. Revenue Risk

Luego encima puedes calcular:

- análisis por canal
- nivel de riesgo
- fugas de revenue
- alertas IA

Ese orden es el correcto.

Qué corrige esto en tu análisis anterior

Hay que ajustar una cosa de lo que habíamos dicho:

Antes te propuse mover tres pantallas a “Riesgo por Propiedad”.

Sigo pensando que **conceptualmente** es correcto, pero viendo lo que ya tienes construido, hay dos opciones válidas:

Opción A — mantenerlas dentro de Revenue Intelligence

Y crear subgrupos visuales:

Revenue Intelligence

- **Configuración**
 - Propiedades
 - Tipos de habitación
 - Calendario de precios
 - Eventos y temporadas
- **Operación y análisis**
 - Importación Revenue
 - Día x Día

- Mensual
- Canales & Segmentos
- Pickup Avanzado
- **Revenue Risk**
 - Análisis por Canal
 - Nivel de Riesgo
 - Fugas de Revenue

Esta opción es la más práctica porque aprovecha lo que ya tienes.

Opción B — sacar “Revenue Risk” a un menú aparte

Es más limpio conceptualmente, pero te obliga a tocar más navegación y puede fragmentar demasiado.

Mi recomendación realista

Opción A.

No tocaría tanto el menú. Reorganizaría internamente el bloque Revenue Intelligence en tres subzonas.

Es más estable y menos destructivo.

Qué implica para Screening / incidencias

Tu observación también refuerza otra cosa:

Si ya tienes toda esta estructura revenue por propiedad, entonces **Screening y incidencias no pueden quedarse en org puro**. Quedarían desalineados con todo el resto del producto.

Porque acabarías teniendo:

- revenue bien imputado por propiedad
- pricing bien imputado por propiedad
- room types bien imputados por propiedad
- eventos bien imputados por propiedad

pero luego:

- incidencias sin propiedad
- screening sin propiedad
- consulta manual sin propiedad

Eso sería incoherente.

Por tanto, tu corrección hace todavía más evidente que:

- consulta automática CSV → selector de propiedad
- consulta manual → selector de propiedad
- registrar incidencia → selector de propiedad

ya no es una mejora, es una obligación para que el sistema no quede partido en dos.

Estructura final recomendada del menú izquierdo

Yo la dejaría así.

OPERATIVA

- Dashboard
- Consulta automática (CSV)
- Consulta manual
- Registrar incidencia

REVENUE INTELLIGENCE

Configuración

- Propiedades
- Tipos de habitación
- Calendario de precios
- Eventos y temporadas

Operación y análisis

- Importación Revenue
- Día x Día
- Mensual
- Canales & Segmentos
- Pickup Avanzado

Revenue Risk

- Análisis por Canal
- Nivel de Riesgo

- Fugas de Revenue

AUDITORÍA

- Estadísticas operativas
- Histórico
- Exportaciones

Esto te deja una estructura coherente sin desmontar lo que ya tienes.

Regla técnica definitiva

La cierro así para que ya no haya ambigüedad:

Regla 1

debacu_eval_properties pertenece al nivel org y es el maestro de propiedades.

Regla 2

Todo maestro derivado de operación hotelera pertenece a una propiedad:

- room types
- calendars
- seasons/events
- pricing rules

Regla 3

Todo dato operativo o analítico real debe asociarse a property_id:

- imports
- incidents
- screening
- risk
- revenue facts
- alerts

Regla 4

Toda vista de grupo puede agregarse por org, pero debe poder discriminar por propiedad.

Orden correcto a partir de aquí

Con esta corrección, el orden de trabajo ya no es “discutir modelo”, sino este:

Paso 1

Cerrar inventario de tablas:

- org-level
- property master
- property facts
- org analytics

Paso 2

Corregir captura operativa:

- CSV screening
- consulta manual
- incidencia

Paso 3

Corregir Revenue Risk:

- que el selector de propiedad filtre de verdad

Paso 4

Corregir auditoría:

- mantener org
- añadir breakdown por property

Mi recomendación práctica ahora

El siguiente paso útil es hacer una **matriz exacta pantalla → nivel → tabla → selector requerido**.

Eso te deja todo aterrizado y evita seguir pensando “esta pantalla quizá era org, quizá propiedad”.

Te la preparo en el siguiente mensaje, ya cerrada, con columnas tipo:

- pantalla
- ámbito correcto
- selector de propiedad sí/no
- tablas afectadas

- Edge Functions a tocar
- prioridad de implementación

Matriz maestra: pantalla → ámbito → selector → datos

1. OPERATIVA

Dashboard

- **Ámbito correcto:** org, con posibilidad de desglose por propiedad
 - **Selector de propiedad:** opcional
 - **Qué debe mostrar:**
 - resumen global del grupo
 - tarjetas con breakdown por propiedad
 - **Tablas:**
 - agregados de consultas
 - agregados de incidencias
 - agregados de screening
 - agregados de revenue risk si aplica
 - **Backend:**
 - endpoint org summary
 - endpoint org by property summary
 - **Prioridad:** media
-

Consulta automática (CSV)

- **Ámbito correcto:** propiedad
- **Selector de propiedad:** obligatorio
- **Qué debe hacer:**
 - elegir propiedad
 - subir CSV
 - procesar screening/import ligado a esa propiedad
- **Tablas:**

- import jobs/runs
 - import rows
 - screening results
 - incidencias derivadas si las generas
 - **Campos mínimos a guardar:**
 - org_id
 - property_id
 - uploaded_by
 - source_file
 - **Backend:**
 - Edge Function de upload/parse/import debe aceptar property_id
 - validar que la propiedad pertenece a la org
 - **Prioridad:** crítica
-

Consulta manual

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio
 - **Qué debe hacer:**
 - lanzar consulta ligada al hotel concreto
 - **Tablas:**
 - screening runs / manual checks
 - results
 - audit trail
 - **Campos mínimos:**
 - org_id
 - property_id
 - performed_by
 - input consultado
 - **Backend:**
 - create manual screening con property_id
 - **Prioridad:** crítica
-

Registrar incidencia

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio
 - **Qué debe hacer:**
 - registrar el hecho en el hotel donde ocurrió
 - **Tablas:**
 - `debacu_evaluations` o equivalente
 - **Campos mínimos:**
 - `org_id`
 - `property_id`
 - `evaluation_date`
 - `channel`
 - `segment`
 - `guest/reservation` si existe
 - **Backend:**
 - create evaluation/incidence con `property_id`
 - **Prioridad:** crítica
-

2. REVENUE INTELLIGENCE — CONFIGURACIÓN

Propiedades

- **Ámbito correcto:** org
- **Selector de propiedad:** no, salvo para elegir activa en cabecera
- **Qué hace:**
 - alta y gestión de propiedades del grupo
- **Tabla principal:**
 - `debacu_eval_properties`
- **Campos clave:**
 - `id`
 - `org_id`
 - `code`
 - `name`

- status
 - timezone
 - **Backend:**
 - CRUD properties
 - validación por org
 - **Prioridad:** ya base maestra
-

Tipos de habitación

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio
 - **Qué hace:**
 - catálogo de room types por hotel
 - **Tablas:**
 - debacu_eval_room_types o equivalente
 - **Campos mínimos:**
 - id
 - org_id
 - property_id
 - code
 - name
 - capacity
 - base_price
 - status
 - **Backend:**
 - CRUD room types filtrado por property_id
 - **Prioridad:** alta
-

Calendario de precios

- **Ámbito correcto:** propiedad
- **Selector de propiedad:** obligatorio
- **Qué hace:**

- precios diarios por room type
 - estancia mínima
 - cierres
 - reglas
 - **Tablas:**
 - price calendar
 - rate rules
 - stay restrictions
 - **Dependencias:**
 - property
 - room types
 - seasons/events
 - **Backend:**
 - read/update by property + date range
 - **Prioridad:** alta
-

Eventos y temporadas

- **Ámbito correcto:** propiedad
- **Selector de propiedad:** obligatorio
- **Qué hace:**
 - define contexto temporal que afecta demanda y pricing
- **Tablas:**
 - seasons
 - events
- **Campos mínimos:**
 - org_id
 - property_id
 - name
 - start_date
 - end_date
 - impact
 - pricing_modifier

- priority
 - **Backend:**
 - CRUD by property
 - **Prioridad:** alta
-

3. REVENUE INTELLIGENCE — OPERACIÓN Y ANÁLISIS

Importación Revenue

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio
 - **Qué hace:**
 - importar CSV PMS/revenue para un hotel concreto
 - **Tablas:**
 - unified import batches
 - unified import rows
 - revenue daily
 - booking lines
 - snapshots
 - **Regla:**
 - 1 import = 1 property
 - **Backend:**
 - import function con `property_id`
 - **Prioridad:** crítica
-

Día x Día

- **Ámbito correcto:** propiedad
- **Selector de propiedad:** obligatorio
- **Qué muestra:**
 - ocupación, ADR, revenue, RevPAR diario de la propiedad
- **Tablas:**
 - revenue_daily

- **Backend:**
 - query by org_id + property_id + range
 - **Prioridad:** alta
-

Mensual

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio
 - **Qué muestra:**
 - agregados mensuales de la propiedad
 - **Tablas:**
 - revenue_daily
 - vistas agregadas mensuales
 - **Backend:**
 - query by property
 - **Prioridad:** alta
-

Canales & Segmentos

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio
 - **Qué muestra:**
 - mix comercial del hotel
 - **Tablas:**
 - revenue by channel
 - revenue by segment
 - **Backend:**
 - query by property
 - **Prioridad:** alta
-

Pickup Avanzado

- **Ámbito correcto:** propiedad
- **Selector de propiedad:** obligatorio

- **Qué muestra:**
 - ritmo de reservas para esa propiedad
 - **Tablas:**
 - pickup snapshots
 - booking lines
 - **Backend:**
 - query by property
 - **Prioridad:** media-alta
-

4. REVENUE INTELLIGENCE — REVENUE RISK

Análisis por Canal

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio y funcional
 - **Qué muestra:**
 - incidencias y net loss por canal de la propiedad
 - **Tablas:**
 - evaluations/incidents con `property_id`
 - revenue correlations si aplica
 - **Backend:**
 - filtrar de verdad por `property_id`
 - **Prioridad:** crítica, porque ahora engaña si el selector no filtra
-

Nivel de Riesgo

- **Ámbito correcto:** propiedad
- **Selector de propiedad:** obligatorio y funcional
- **Qué muestra:**
 - distribución de riesgo de esa propiedad
- **Tablas:**
 - evaluations / risk aggregates
- **Backend:**
 - query by property

- **Prioridad:** crítica
-

Fugas de Revenue

- **Ámbito correcto:** propiedad
 - **Selector de propiedad:** obligatorio y funcional
 - **Qué muestra:**
 - pérdida o exposición económica del hotel
 - **Tablas:**
 - incidents/evaluations
 - revenue loss views
 - alerts if applicable
 - **Backend:**
 - query by property
 - **Prioridad:** crítica
-

5. AUDITORÍA

Estadísticas operativas

- **Ámbito correcto:** org
- **Selector de propiedad:** no obligatorio; opcional como filtro secundario
- **Qué muestra:**
 - total consultas grupo
 - total registros añadidos grupo
 - riesgo agregado grupo
 - breakdown por propiedad
- **Tablas:**
 - consultas
 - incidencias
 - audit logs
 - screening runs
- **IA aquí:**
 - sí, pero como resumen comparativo entre propiedades

- **Backend:**
 - endpoint org summary + breakdown per property
 - **Prioridad:** media-alta
-

Histórico

- **Ámbito correcto:** org
 - **Selector de propiedad:** filtro opcional
 - **Qué muestra:**
 - trazabilidad del grupo
 - **Tablas:**
 - audit history
 - action logs
 - **Cada fila debería tener:**
 - org_id
 - property_id cuando aplique
 - usuario
 - acción
 - fecha
 - **Backend:**
 - list history with property filter
 - **Prioridad:** media
-

Exportaciones

- **Ámbito correcto:** org con opción property
- **Selector de propiedad:** depende del scope
- **Scopes correctos:**
 - ORG
 - PROPERTY
- **Qué hace:**
 - export global del grupo
 - export por propiedad

- export org con columna property
 - **Tablas:**
 - export jobs
 - export artifacts
 - audit export logs
 - **Backend:**
 - export builder con scope_type + property_id?
 - **Prioridad:** media-alta
-

Matriz resumida de selectores

Selector obligatorio de propiedad

- Consulta automática CSV
- Consulta manual
- Registrar incidencia
- Tipos de habitación
- Calendario de precios
- Eventos y temporadas
- Importación Revenue
- Día x Día
- Mensual
- Canales & Segmentos
- Pickup Avanzado
- Análisis por Canal
- Nivel de Riesgo
- Fugas de Revenue

Selector no obligatorio

- Dashboard
- Estadísticas operativas
- Histórico
- Exportaciones

- Propiedades
-

Inventario técnico por nivel

Nivel org

- debacu_eval_organizations
- debacu_eval_org_members
- billing/subscriptions/seats
- debacu_eval_properties
- audit/export headers globales

Nivel property master

- room types
- seasons
- events
- pricing rules
- price calendar

Nivel property facts

- revenue imports
- revenue daily
- pickup snapshots
- booking lines
- manual screenings
- CSV screenings
- evaluations/incidents
- revenue alerts

Nivel org analytics

- audit stats
- historical activity
- cross-property insights

- exports globales
-

Qué Edge Functions tocar primero

Prioridad 1 — no seguir generando datos ambiguos

1. consulta automática CSV
2. consulta manual
3. registrar incidencia
4. importación revenue

Estas cuatro deben exigir property_id.

Prioridad 2 — corregir pantallas que hoy parecen por propiedad pero no filtran bien

5. análisis por canal
6. nivel de riesgo
7. fugas de revenue

Prioridad 3 — consolidar reporting corporativo

8. estadísticas operativas
 9. histórico
 10. exportaciones
-

Qué tablas revisar primero

Te diría que la revisión empiece por estas familias:

Familia 1 — incidencias / risk

- debacu_evaluations
- tablas derivadas de scoring/risk
- alert tables

Familia 2 — screening

- screening runs

- screening results
- import jobs
- import rows

Familia 3 — revenue facts

- revenue imports
- revenue daily
- booking lines
- pickup snapshots
- revenue alerts

Familia 4 — masters de property

- properties
- room types
- price calendar
- seasons/events

Familia 5 — auditoría

- audit history
 - export logs
 - stats source tables
-

Decisiones cerradas que ya no tocaría

Decisión 1

Propiedades queda como maestro org.

Decisión 2

Todo hecho operativo real lleva `property_id`.

Decisión 3

Todo selector de propiedad que exista en UI debe filtrar de verdad.

Decisión 4

Exportaciones, histórico y estadísticas operativas pueden abrir a nivel org, pero con desglose por property.

Decisión 5

1 CSV operativo / revenue por ahora = 1 property.

Siguiente paso correcto

Ahora ya no toca seguir diseñando. Toca bajar esto a ejecución.

El orden bueno es:

1. revisar tablas afectadas
2. marcar cuáles necesitan `property_id` obligatorio
3. hacer migraciones
4. tocar Edge Functions críticas
5. luego arreglar frontend

En el siguiente mensaje te preparo el **plan SQL + backend**, empezando por una lista concreta de tablas que, por lo que lleváis construido, casi seguro hay que tocar primero.

Ir pantalla por pantalla es más seguro que tocar el modelo entero de golpe, porque así cerramos cada flujo completo:

- UI
- selector org/property
- payload al Edge Function
- validación backend
- tablas afectadas
- lectura posterior
- y cómo queda en auditoría

Eso evita romper cosas que ya funcionan.

Forma de trabajo recomendada

Para cada pantalla, me pasas:

1. Frontend

El fichero completo de la pantalla o componente principal.

2. Edge Function

La función o funciones que usa esa pantalla:

- lectura
- creación
- actualización
- importación
- exportación

3. Si aplica, tablas implicadas

Aunque sea solo una lista rápida:

- nombre de tabla
- campos principales
- y si ya tiene `property_id` o no

4. Qué comportamiento quieres exactamente

Por ejemplo:

- “aquí el selector de propiedad debe ser obligatorio”
 - “aquí debe abrir por org pero con filtro por propiedad”
 - “aquí ahora lee por org y quiero que pase a property”
-

Cómo lo cerraremos en cada pantalla

Yo te devolveré cada una con esta estructura:

A. Diagnóstico

Qué está mal o ambiguo ahora mismo.

B. Modelo correcto

Si esa pantalla debe ser:

- org
- property
- org con breakdown property

C. Cambios frontend

Qué hay que añadir o quitar:

- selector

- estado
- payload
- validaciones
- textos

D. Cambios Edge Function

Qué debe recibir:

- org_id
- property_id
- scope
- etc.

Y qué debe validar.

E. Cambios de BD

Si hace falta:

- añadir property_id
- índices
- constraints
- backfill
- vistas

F. Orden de implementación

Para que no se rompa nada.

Orden que te recomiendo seguir

Empezaría por las pantallas que hoy generan datos ambiguos.

Bloque 1 — crítico

1. **Consulta automática CSV**
2. **Consulta manual**
3. **Registrar incidencia**

Porque esas tres crean el dato base.

Bloque 2 — revenue risk

4. **Análisis por Canal**

5. Nivel de Riesgo

6. Fugas de Revenue

Porque ahora aparentan ser por propiedad pero no lo son del todo.

Bloque 3 — revenue operativo

7. Importación Revenue

8. Día x Día

9. Mensual

10. Canales & Segmentos

11. Pickup Avanzado

Bloque 4 — maestros

12. Propiedades

13. Tipos de habitación

14. Calendario de precios

15. Eventos y temporadas

Bloque 5 — auditoría

16. Estadísticas operativas

17. Histórico

18. Exportaciones

Qué necesito en el próximo mensaje

Pásame la primera pantalla con este formato, aunque sea simple:

- nombre de pantalla
- fichero frontend completo
- Edge Function/s que usa
- tablas que toca si las sabes
- qué quieres que haga exactamente

Por ejemplo:

- “Pantalla: Consulta automática CSV”
- pegas el .tsx
- pegas la Edge Function

- me dices: “quiero selector obligatorio de propiedad y que todo quede imputado a esa property”

Y arrancamos por ahí.

PANTALLA DE IMPORTACION CSV AUTOMATICO CONSULTAS RIESGO

PRIMERA PANTALLA QUE TE PASO CONSULTA AUTOMATICO CSV ESTA //

```
src/components/ScreeningCsvPage.tsx import React, { useEffect, useMemo, useState } from
"react"; import { Box, Button, Divider, Drawer, Stack, Typography } from "@mui/material"; import
RunsList from "@components/RunsList"; import ImportWizard from
"@components/ImportWizard"; import RunDetail from "@components/RunDetail"; import
ImportJobsDialog from "@components/ImportJobsDialog"; import { useScreeningRuns } from
"@/hooks/useScreeningRuns"; type Props = { orgId: string }; export default function
ScreeningCsvPage({ orgId }: Props) { const [selectedRunId, setSelectedRunId] = useState<string |
null>(null); const [openWizard, setOpenWizard] = useState(false); const [openRuns, setOpenRuns]
= useState(false); const [openImports, setOpenImports] = useState(false); const canUse =
useMemo(() => String(orgId || "").trim().length > 0, [orgId]); // ☒ Cargar runs aquí solo para auto-
seleccionar el último const { runs } = useScreeningRuns(orgId, 1); useEffect(() => { if (!canUse)
{ setSelectedRunId(null); return; } // si aún no hay selección, pon el último run automáticamente if
(!selectedRunId && runs && runs.length > 0) { setSelectedRunId(runs[0].id); } }, [canUse, runs,
selectedRunId]); return ( <Box sx={{ p: 2 }}> <Stack direction="row" alignItems="center"
justifyContent="space-between" spacing={2}> <Box> <Typography variant="h6">Screening por
CSV</Typography> <Typography variant="body2" color="text.secondary"> Importa un CSV,
valida (dry-run) y genera un run (persona + fecha). </Typography> </Box> <Stack direction="row"
spacing={1}> <Button variant="outlined" onClick={() => setOpenRuns(true)} disabled={!
canUse}> Ver runs </Button> <Button variant="outlined" onClick={() => setOpenImports(true)}
disabled={!canUse}> Ver importaciones </Button> <Button variant="contained" onClick={() =>
setOpenWizard(true)} disabled={!canUse}> Nuevo screening (CSV) </Button> </Stack> </Stack>
<Divider sx={{ my: 2 }} /> { /* ☒ A ancho completo: aquí va la tabla + alertas */} <RunDetail
orgId={orgId} runId={selectedRunId} /> { /* Drawer de Runs */} <Drawer anchor="left"
open={openRuns} onClose={() => setOpenRuns(false)} PaperProps={{ sx: { width: 460, p: 2 } }}
> <RunsList orgId={orgId} selectedRunId={selectedRunId} onSelectRun={(id) =>
{ setSelectedRunId(id); setOpenRuns(false); }} onCreateNew={() => { setOpenRuns(false);
setOpenWizard(true); }} /> </Drawer> { /* Dialog de Importaciones */} <ImportJobsDialog
open={openImports} orgId={orgId} onClose={() => setOpenImports(false)} /> <ImportWizard
open={openWizard} orgId={orgId} onClose={() => setOpenWizard(false)} onCommitted={(runId)
=> { setOpenWizard(false); setSelectedRunId(runId); }} /> </Box> ); } Y USA ESTAS //
```

```
src/components/RunsList.tsx import React, { useMemo } from "react"; import { Box, Button, Card,
CardContent, CircularProgress, Divider, List, ListItemButton, ListItemText, Stack, Typography, }
from "@mui/material"; import { useScreeningRuns } from "@/hooks/useScreeningRuns"; import
type { ScreeningRun } from "@types/screeningCsv.types"; type Props = { orgId: string;
selectedRunId: string | null; onSelectRun: (runId: string) => void; onCreateNew: () => void; };
function fmtTs(ts?: string | null) { if (!ts) return ""; const d = new Date(ts); if
(Number.isNaN(d.getTime())) return ts; return d.toLocaleString(); } function runLabel(r:
ScreeningRun) { const t = String(r.run_type || "").toUpperCase(); return t || "RUN"; } export default
function RunsList({ orgId, selectedRunId, onSelectRun, onCreateNew }: Props) { const { loading,
error, runs, reload } = useScreeningRuns(orgId, 80); const hasRuns = useMemo(() => (runs?.length
```

```

|| 0) > 0, [runs]); return ( <Card variant="outlined"> <CardContent> <Stack direction="row"
alignItems="center" justifyContent="space-between" spacing={1}> <Typography
variant="subtitle1">Runs</Typography> <Stack direction="row" spacing={1}> <Button
size="small" onClick={reload} disabled={loading}> Refrescar </Button> <Button size="small"
variant="contained" onClick={onCreateNew}> Nuevo </Button> </Stack> </Stack> <Divider
sx={{ my: 1 }} /> {loading && ( <Box sx={{ py: 2, display: "flex", alignItems: "center", gap:
1 }}> <CircularProgress size={18} /> <Typography variant="body2">Cargando
runs...</Typography> </Box> )} {!loading && error && ( <Box sx={{ py: 1 }}> <Typography
variant="body2" color="error"> {error} </Typography> </Box> )} {!loading && !error && !
hasRuns && ( <Box sx={{ py: 2 }}> <Typography variant="body2" color="text.secondary"> No
hay runs todavía. Crea uno con “Nuevo”. </Typography> </Box> )} {!loading && !error &&
hasRuns && ( <List dense disablePadding sx={{ maxHeight: 620, overflow: "auto" }}>
{runs.map((r) => { const selected = selectedRunId === r.id; const total =
Number(r.total_analyzed ?? 0); const high = Number(r.high_count ?? 0); const med =
Number(r.medium_count ?? 0); const low = Number(r.low_count ?? 0); return ( <ListItemButton
key={r.id} selected={selected} onClick={() => onSelectRun(r.id)} sx={{ borderRadius: 1, mb: 0.5,
border: "1px solid", borderColor: selected ? "primary.main" : "divider", }} > <ListItemText
primary={ <Stack direction="row" justifyContent="space-between" spacing={1}> <Typography
variant="body2" sx={{ fontWeight: 700 }}> {runLabel(r)} </Typography> <Typography
variant="caption" color="text.secondary"> {fmtTs(r.created_at)} </Typography> </Stack> }
secondary={ <Stack direction="row" spacing={1} sx={{ mt: 0.5, flexWrap: "wrap" }}>
<Typography variant="caption" color="text.secondary"> Total: {total} </Typography>
<Typography variant="caption" color="text.secondary"> HIGH: {high} </Typography>
<Typography variant="caption" color="text.secondary"> MED: {med} </Typography>
<Typography variant="caption" color="text.secondary"> LOW: {low} </Typography>
</Stack> } /> </ListItemButton> ); }} </List> )} </CardContent> </Card> ); } Y DESTA //
src/components/RunDetail.tsx import React, { useMemo, useState } from "react"; import { Alert,
Box, Card, CardContent, Chip, Divider, MenuItem, Stack, TextField, Typography, } from
"@mui/material"; import { useRunDetail } from "@hooks/useRunDetail"; import ResultsTable
from "@components/ResultsTable"; import AlertsPanel from "@components/AlertsPanel"; type
Props = { orgId: string; runId: string | null; }; function fmtTs(ts?: string | null) { if (!ts) return "";
const d = new Date(ts); if (Number.isNaN(d.getTime())) return ts; return d.toLocaleString(); }
export default function RunDetail({ orgId, runId }: Props) { // orgId se mantiene por
consistencia/posibles usos futuros (por ahora no hace falta aquí) void orgId; const [riskBand,
setRiskBand] = useState<string>(""); const [onlyChanged, setOnlyChanged] =
useState<boolean>(false); const [unresolvedAlertsOnly, setUnresolvedAlertsOnly] =
useState<boolean>(false); const { loading, error, run, results, alerts, reload } = useRunDetail(runId,
{ riskBand: riskBand || undefined, onlyChanged: onlyChanged || undefined, unresolvedAlertsOnly:
unresolvedAlertsOnly || undefined, }); const kpis = useMemo(() => { const total =
Number(run?.total_analyzed ?? 0); const high = Number(run?.high_count ?? 0); const med =
Number(run?.medium_count ?? 0); const low = Number(run?.low_count ?? 0); return { total, high,
med, low }; }, [run]); if (!runId) { return ( <Card variant="outlined"> <CardContent> <Typography
variant="body2" color="text.secondary"> Selecciona un run. </Typography> </CardContent>
</Card> ); } return ( <Stack spacing={2}> <Card variant="outlined"> <CardContent> <Stack
direction="row" justifyContent="space-between" alignItems="flex-start" spacing={2}> <Box>
<Typography variant="subtitle1"> Run: {String(run?.run_type || "").toUpperCase()}
</Typography> <Typography variant="caption" color="text.secondary"> {run?.id || runId}
{run?.created_at ? ` · ${fmtTs(run.created_at)} : ` : ""} </Typography> </Box> <Chip
label={loading ? "Cargando..." : "Refrescar"} onClick={() => !loading && reload()}
color={loading ? "default" : "primary"} variant={loading ? "outlined" : "filled"} clickable={!
loading} /> </Stack> <Divider sx={{ my: 1 }} /> {error && ( <Alert severity="error" sx={{ mb:
1 }}> {error} </Alert> )} <Stack direction={{ xs: "column", md: "row" }} spacing={1}> <Chip

```



```

label={Total: ${kpis.total}} variant="outlined" /> <Chip label={HIGH: $
{kpis.high}} color="error" variant="outlined" /> <Chip label={MED: ${kpis.med}}
color="warning" variant="outlined" /> <Chip label={LOW: ${kpis.low}} color="success"
variant="outlined" /> <Chip label={Alertas: ${alerts.length}} variant="outlined" />
</Stack> <Divider sx={{ my: 2 }} /> <Stack direction={{ xs: "column", md: "row" }} spacing={2}
sx={{ alignItems: { xs: "stretch", md: "center" } }} > <TextField select size="small" label="Filtrar
por risk_band" value={riskBand} onChange={(e) => setRiskBand(e.target.value)}
sx={{ minWidth: 220 }} > <MenuItem value="">(Todos)</MenuItem> <MenuItem
value="HIGH">HIGH</MenuItem> <MenuItem value="MEDIUM">MEDIUM</MenuItem>
<MenuItem value="LOW">LOW</MenuItem> </TextField> <TextField select size="small"
label="Solo cambios de banda" value={onlyChanged ? "YES" : "NO"} onChange={(e) =>
setOnlyChanged(e.target.value === "YES")} sx={{ minWidth: 220 }} > <MenuItem
value="NO">No</MenuItem> <MenuItem value="YES">Sí</MenuItem> </TextField> <TextField
select size="small" label="Alertas sin resolver" value={unresolvedAlertsOnly ? "YES" : "NO"}
onChange={(e) => setUnresolvedAlertsOnly(e.target.value === "YES")} sx={{ minWidth: 220 }}
> <MenuItem value="NO">No</MenuItem> <MenuItem value="YES">Sí</MenuItem>
</TextField> </Stack> </CardContent> </Card> { /* ☒ Layout pro: resultados a ancho flexible +
alertas ancho fijo */ <Stack direction={{ xs: "column", lg: "row" }} spacing={2}
alignItems="flex-start" sx={{ width: "100%" }} > { /* IMPORTANTÍSIMO: minWidth: 0 para que
el overflowX funcione en flex */ <Box sx={{ flex: 1, width: "100%", minWidth: 0 }} > { /* Scroll
horizontal SOLO dentro de resultados */ <Box sx={{ width: "100%", overflowX: "auto" }} > { /*
Si tu tabla es muy ancha, fuerza que tenga "mínimo contenido" */ <Box sx={{ minWidth: 900 }} >
<ResultsTable loading={loading} results={results} /> </Box> </Box> </Box> <Box sx={{ width:
{ xs: "100%", lg: 380 }, flexShrink: 0 }} > <AlertsPanel loading={loading} alerts={alerts} />
</Box> </Stack> </Stack> ); } // src/components/ImportWizard.tsx import React, { useEffect,
useMemo, useState } from "react"; import { Alert, Box, Button, Card, CardContent,
CircularProgress, Dialog, DialogActions, DialogContent, DialogTitle, Divider, MenuItem, Stack,
TextField, Typography, } from "@mui/material"; import { useImportProfiles } from
"@/hooks/useImportProfiles"; import type { ImportProfile,
ImportValidateCommitCommitResponse, ImportValidateCommitDryRunResponse, } from
"@/types/screeningCsv.types"; import { importValidateCommit, uploadScreeningCsvToStorage, }
from "@/services/screeningCsv.service"; type Props = { open: boolean; orgId: string; onClose: ()
=> void; onCommitted: (runId: string) => void; }; type Step = "SETUP" | "DRYRUN_DONE" |
"COMMIT_DONE"; function pickRunType(profile: ImportProfile) { // Por defecto: usamos
source_type como run_type (tú ya lo haces en backend si no viene) return
String(profile.source_type || "").toUpperCase(); } export default function ImportWizard({ open,
orgId, onClose, onCommitted }: Props) { const { loading: loadingProfiles, error: profilesError,
profiles, reload } = useImportProfiles(orgId); const [profileId, setProfileId] = useState<string>("");
const [file, setFile] = useState<File | null>(null); const [step, setStep] = useState<Step>("SETUP");
const [busy, setBusy] = useState(false); const [err, setErr] = useState<string | null>(null); const
[dryRun, setDryRun] = useState<ImportValidateCommitDryRunResponse | null>(null); const
[commitRes, setCommitRes] = useState<ImportValidateCommitCommitResponse | null>(null);
const selectedProfile = useMemo( () => profiles.find((p) => p.id === profileId) || null, [profiles,
profileId], ); const runType = useMemo(() => { if (!selectedProfile) return ""; return
pickRunType(selectedProfile); }, [selectedProfile]); const canDryRun = useMemo(() => { return
String(orgId || "").trim() && String(profileId || "").trim() && !file && !busy; }, [orgId, profileId,
file, busy]); const canCommit = useMemo(() => { return !dryRun && !busy; }, [dryRun, busy]);
useEffect(() => { if (!open) return; // reset wizard on open setStep("SETUP"); setBusy(false);
setErr(null); setDryRun(null); setCommitRes(null); // si hay perfiles, selecciona el primero if
(profiles?.length && !profileId) { setProfileId(profiles[0].id); } // eslint-disable-next-line react-
hooks/exhaustive-deps }, [open]); useEffect(() => { // auto pick first profile when loaded if (open
&& !profileId && profiles?.length) setProfileId(profiles[0].id); }, [open, profiles, profileId]); async

```

```

function handleDryRun() { if (!canDryRun || !selectedProfile || !file) return; setBusy(true);
setErr(null); try { // 1) upload file to Storage const filePath = await
uploadScreeningCsvToStorage(orgId, file); // 2) dry-run const res = await
importValidateCommit({ orgId, profileId: selectedProfile.id, runType, dryRun: true, filePath, }); if
(res.mode !== "DRY_RUN") throw new Error("unexpected_response_mode"); setDryRun(res);
setStep("DRYRUN_DONE"); } catch (e: any) { setErr(String(e?.message || e)); } finally
{ setBusy(false); } } async function handleCommit() { if (!canCommit || !selectedProfile) return;
setBusy(true); setErr(null); try { // Importante: el commit vuelve a crear import_job y run. // Para no
depender del filePath anterior, volvemos a subir el CSV // (simple y robusto; si luego quieres
optimizar, pasamos import_job_id). if (!file) throw new Error("missing_file"); const filePath =
await uploadScreeningCsvToStorage(orgId, file); const res = await importValidateCommit({ orgId,
profileId: selectedProfile.id, runType, dryRun: false, filePath, }); if (res.mode !== "COMMIT")
throw new Error("unexpected_response_mode"); setCommitRes(res);
setStep("COMMIT_DONE"); // refrescar runs list en la página (si el padre recarga)
onCommitted(res.run_id); } catch (e: any) { setErr(String(e?.message || e)); } finally
{ setBusy(false); } } const errorsPreview = dryRun?.errors?.slice(0, 20) || []; const previewRows =
dryRun?.preview?.slice(0, 10) || []; return ( <Dialog open={open} onClose={onClose} fullWidth
maxWidth="md"> <DialogTitle>Nuevo Screening (CSV)</DialogTitle> <DialogContent dividers>
<Stack spacing={2}> {profilesError && ( <Alert severity="error"> Error cargando perfiles:
{profilesError}{" "} <Button size="small" onClick={reload} sx={{ ml: 1 }}> Reintentar </Button>
</Alert> )} {err && <Alert severity="error">{err}</Alert> } <Card variant="outlined">
<CardContent> <Typography variant="subtitle2">Configuración</Typography> <Divider
sx={{ my: 1 }} /> <Stack direction={{ xs: "column", md: "row" }} spacing={2}> <TextField select
label="Perfil de importación" size="small" fullWidth value={profileId} onChange={(e) =>
setProfileId(e.target.value)} disabled={busy || loadingProfiles} > {profiles.map((p) =>
( <MenuItem key={p.id} value={p.id}> {p.name} — {String(p.source_type || "").toUpperCase()}
</MenuItem> ))) </TextField> <TextField label="run_type efectivo" size="small" fullWidth
value={runType} disabled /> </Stack> <Box sx={{ mt: 2 }}> <Button variant="outlined"
component="label" disabled={busy} > Seleccionar CSV <input type="file" hidden
accept=".csv,text/csv,text/plain" onChange={(e) => { const f = e.target.files?.[0] || null; setFile(f);
setDryRun(null); setCommitRes(null); setStep("SETUP"); } } /> </Button> <Typography
variant="caption" sx={{ ml: 2 }} color="text.secondary"> {file ? file.name : "Ningún archivo
seleccionado"} </Typography> </Box> <Box sx={{ mt: 2 }}> <Button variant="contained"
onClick={handleDryRun} disabled={!canDryRun} > {busy ? "Procesando..." : "Validar (Dry-
run)"} </Button> {busy && ( <CircularProgress size={18} sx={{ ml: 2, verticalAlign:
"middle" }} /> )} </Box> </CardContent> </Card> {dryRun && ( <Card variant="outlined">
<CardContent> <Stack direction="row" justifyContent="space-between" alignItems="center">
<Typography variant="subtitle2">Resultado Dry-run</Typography> <Typography
variant="caption" color="text.secondary"> import_job_id: {dryRun.import_job_id} </Typography>
</Stack> <Divider sx={{ my: 1 }} /> <Stack direction={{ xs: "column", md: "row" }}
spacing={2}> <Box> <Typography variant="body2">Total filas:
{dryRun.total_rows}</Typography> <Typography variant="body2">Válidas:
{dryRun.valid_rows}</Typography> <Typography variant="body2">Inválidas:
{dryRun.invalid_rows}</Typography> </Box> <Box sx={{ flex: 1 }}> <Typography
variant="body2" sx={{ fontWeight: 700 }}> Errores (primeros {errorsPreview.length})
</Typography> {errorsPreview.length === 0 ? ( <Typography variant="body2"
color="text.secondary"> Sin errores. </Typography> ) : ( <Box component="ul" sx={{ m: 0, pl:
2 }}> {errorsPreview.map((e, idx) => ( <li key={idx}> <Typography variant="caption"> Row
{e.row} {e.field ? ({e.field}) : ""}: {e.error} </Typography> </li> ))) </Box> )} </Box>
</Stack> <Divider sx={{ my: 1 }} /> <Typography variant="body2" sx={{ fontWeight: 700 }}>
Preview (primeras {previewRows.length}) </Typography> {previewRows.length === 0 ?
( <Typography variant="body2" color="text.secondary"> Sin preview. </Typography> ) : ( <Box

```

```

sx={{ mt: 1, p: 1, borderRadius: 1, border: "1px solid", borderColor: "divider", backgroundColor:
"background.default", maxHeight: 220, overflow: "auto", fontFamily: "monospace", fontSize: 12,
whiteSpace: "pre", }} > {JSON.stringify(previewRows, null, 2)} </Box> )} <Box sx={{ mt: 2 }}>
<Button variant="contained" color="success" onClick={handleCommit} disabled={!canCommit} >
{busy ? "Procesando..." : "Confirmar (Commit)"} </Button> </Box> </CardContent> </Card> )}
{commitRes && ( <Alert severity="success"> Commit OK. Run creado: {commitRes.run_id}
(HIGH={commitRes.high} / MED={commitRes.medium} / LOW={commitRes.low}) </Alert> )}
</Stack> </DialogContent> <DialogActions> <Button onClick={onClose} disabled={busy}>
Cerrar </Button> </DialogActions> </Dialog> ); } Y ESTA // src/components/ImportJobsDialog.tsx
import React, { useMemo, useState } from "react"; import { Box, Button, Chip, Dialog,
DialogActions, DialogContent, DialogTitle, Divider, IconButton, MenuItem, Stack, Table,
TableBody, TableCell, TableHead, TableRow, TextField, Tooltip, Typography, CircularProgress, }
from "@mui/material"; import CloseIcon from "@mui/icons-material/Close"; import
{ useImportJobs } from "@/hooks/useImportJobs"; type Props = { open: boolean; onClose: () =>
void; orgId: string; }; function isoDateOnly(d: Date) { // yyyy-mm-dd const y = d.getFullYear();
const m = String(d.getMonth() + 1).padStart(2, "0"); const day = String(d.getDate()).padStart(2,
"0"); return `${y}-${m}-${day}`; } function startOfDayISO(dateOnly: string) { // dateOnly =
yyyy-mm-dd -> ISO local-ish; para filtros vale con Z return $
{dateOnly}T00:00:00.000Z; } function endOfDayISO(dateOnly: string) { return $
{dateOnly}T23:59:59.999Z; } function fmtTs(ts?: string | null) { if (!ts) return "-"; const d =
new Date(ts); if (Number.isNaN(d.getTime())) return ts; return d.toLocaleString(); } function
statusChipColor(status: string) { const s = String(status || "").toUpperCase(); if (s ===
"COMMITTED") return "success"; if (s === "VALIDATED") return "info"; if (s === "FAILED")
return "error"; if (s === "UPLOADED") return "warning"; return "default"; } export default
function ImportJobsDialog({ open, onClose, orgId }: Props) { const [rangePreset, setRangePreset]
= useState<"WEEK" | "MONTH" | "CUSTOM">("MONTH"); const [status, setStatus] =
useState<string>("ALL"); const [runType, setRunType] = useState<string>("ALL"); const today =
useMemo(() => new Date(), []); const defaultFrom = useMemo(() => { const d = new Date(today);
d.setDate(d.getDate() - 30); return isoDateOnly(d); }, [today]); const defaultFromWeek =
useMemo(() => { const d = new Date(today); d.setDate(d.getDate() - 7); return isoDateOnly(d); },
[today]); const [fromDate, setFromDate] = useState<string>(defaultFrom); const [toDate,
setToDate] = useState<string>(isoDateOnly(today)); const effectiveFrom = useMemo(() => { if
(rangePreset === "WEEK") return startOfDayISO(defaultFromWeek); if (rangePreset ===
"MONTH") return startOfDayISO(defaultFrom); return fromDate ? startOfDayISO(fromDate) :
undefined; }, [rangePreset, defaultFrom, defaultFromWeek, fromDate]); const effectiveTo =
useMemo(() => { if (rangePreset === "WEEK") return endOfDayISO(isoDateOnly(today)); if
(rangePreset === "MONTH") return endOfDayISO(isoDateOnly(today)); return toDate ?
endOfDayISO(toDate) : undefined; }, [rangePreset, today, toDate]); const { loading, error, jobs,
reload } = useImportJobs(orgId, { from: effectiveFrom, to: effectiveTo, status, runType, limit:
200, }); const totals = useMemo(() => { const total = jobs.length; const committed = jobs.filter((j)
=> String(j.status || "").toUpperCase() === "COMMITTED").length; const failed = jobs.filter((j) =>
String(j.status || "").toUpperCase() === "FAILED").length; return { total, committed, failed }; },
[jobs]); return ( <Dialog open={open} onClose={onClose} fullWidth maxWidth="lg">
<DialogTitle sx={{ pr: 6 }}> Importaciones (CSV) <IconButton onClick={onClose}
sx={{ position: "absolute", right: 8, top: 8 }} aria-label="Cerrar" > <CloseIcon /> </IconButton>
</DialogTitle> <DialogContent> <Stack direction={{ xs: "column", md: "row" }} spacing={1.5}
alignItems={{ xs: "stretch", md: "center" }}> <TextField select size="small" label="Rango"
value={rangePreset} onChange={(e) => setRangePreset(e.target.value as any)} sx={{ minWidth:
180 }} > <MenuItem value="WEEK">Semana (últimos 7 días)</MenuItem> <MenuItem
value="MONTH">Mes (últimos 30 días)</MenuItem> <MenuItem
value="CUSTOM">Personalizado</MenuItem> </TextField> {rangePreset === "CUSTOM" &&
( << <TextField size="small" type="date" label="Desde" value={fromDate} onChange={(e) =>

```

```

setFromDate(e.target.value)} InputLabelProps={{ shrink: true }} sx={{ minWidth: 160 }} />
<TextField size="small" type="date" label="Hasta" value={toDate} onChange={(e) =>
setToDate(e.target.value)} InputLabelProps={{ shrink: true }} sx={{ minWidth: 160 }} /> </> )}
<TextField select size="small" label="Estado" value={status} onChange={(e) =>
setStatus(e.target.value)} sx={{ minWidth: 160 }} > <MenuItem
value="ALL">(Todos)</MenuItem> <MenuItem
value="UPLOADED">UPLOADED</MenuItem> <MenuItem
value="VALIDATED">VALIDATED</MenuItem> <MenuItem
value="COMMITTED">COMMITTED</MenuItem> <MenuItem
value="FAILED">FAILED</MenuItem> </TextField> <TextField select size="small" label="Run
type" value={runType} onChange={(e) => setRunType(e.target.value)} sx={{ minWidth: 220 }} >
<MenuItem value="ALL">(Todos)</MenuItem> <MenuItem
value="FUTURE_BOOKINGS">FUTURE_BOOKINGS</MenuItem> <MenuItem
value="INHOUSE_TODAY">INHOUSE_TODAY</MenuItem> <MenuItem
value="HISTORICAL_STAYS">HISTORICAL_STAYS</MenuItem> <MenuItem
value="HISTORICAL_BOOKINGS">HISTORICAL_BOOKINGS</MenuItem> </TextField>
<Box sx={{ flex: 1 }} /> <Stack direction="row" spacing={1} alignItems="center"
justifyContent="flex-end"> <Chip size="small" label={Total: ${totals.total}}
variant="outlined" /> <Chip size="small" label={OK: ${totals.committed}}
color="success" variant="outlined" /> <Chip size="small" label={FAILED: $
${totals.failed}} color="error" variant="outlined" /> <Button size="small"
onClick={reload} disabled={loading}> Refrescar </Button> </Stack> </Stack> <Divider
sx={{ my: 2 }} /> {loading && ( <Box sx={{ py: 2, display: "flex", alignItems: "center", gap:
1 }}> <CircularProgress size={18} /> <Typography variant="body2">Cargando importaciones...
</Typography> </Box> )} {!loading && error && ( <Typography variant="body2" color="error">
{error} </Typography> )} {!loading && !error && jobs.length === 0 && ( <Typography
variant="body2" color="text.secondary"> No hay importaciones para este filtro. </Typography> )}
{!loading && !error && jobs.length > 0 && ( <Box sx={{ border: "1px solid", borderColor:
"divider", borderRadius: 1, overflow: "auto", maxHeight: 620 }}> <Table size="small"
stickyHeader> <TableHead> <TableRow> <TableCell>Fecha</TableCell> <TableCell>Run
type</TableCell> <TableCell>Estado</TableCell> <TableCell align="right">Total</TableCell>
<TableCell align="right">Válidas</TableCell> <TableCell align="right">Inválidas</TableCell>
<TableCell>Archivo</TableCell> <TableCell>ID</TableCell> </TableRow> </TableHead>
<TableBody> {jobs.map((j) => { const st = String(j.status || "").toUpperCase(); const rt =
String(j.run_type || "").toUpperCase(); return ( <TableRow key={j.id} hover>
<TableCell>{fmtTs(j.created_at)}</TableCell> <TableCell> <Typography variant="body2"
sx={{ fontWeight: 700 }}> {rt || "-"} </Typography> </TableCell> <TableCell> <Chip
size="small" label={st || "-"} color={statusChipColor(st) as any} variant="outlined" />
</TableCell> <TableCell align="right">{Number(j.total_rows ?? 0)}</TableCell> <TableCell
align="right">{Number(j.valid_rows ?? 0)}</TableCell> <TableCell
align="right">{Number(j.invalid_rows ?? 0)}</TableCell> <TableCell sx={{ maxWidth: 420 }}>
<Tooltip title={j.file_path || "-"}> <Typography variant="body2" noWrap> {j.file_path || "-"}
</Typography> </Tooltip> </TableCell> <TableCell> <Tooltip title={j.id}> <Typography
variant="body2" sx={{ fontFamily: "monospace" }}> {j.id.slice(0, 8)}... </Typography>
</Tooltip> </TableCell> </TableRow> ); })} </TableBody> </Table> </Box> )} </DialogContent>
<DialogActions> <Button onClick={onClose}>Cerrar</Button> </DialogActions> </Dialog> ); }
Perfecto. Esta primera pantalla ya deja claro el problema.

```

Diagnóstico de esta pantalla

Ahora mismo **Consulta automática (CSV)** está modelada a nivel **org** y no a nivel **property**.

Se ve en todos los componentes:

- `ScreeningCsvPage({ orgId })`
- `RunsList` carga runs por `orgId`
- `ImportWizard` sube/importa por `orgId`
- `ImportJobsDialog` lista jobs por `orgId`
- `RunDetail` recibe `orgId`, pero en realidad trabaja por `runId`

Qué falla funcionalmente

Si una org tiene 2 propiedades:

- no sabes para qué propiedad subes el CSV
- el import job queda ambiguo
- el run queda ambiguo
- los resultados quedan mezclados a nivel grupo
- luego no puedes explotar bien por hotel
- auditoría e IA pierden precisión

Así que esta pantalla **debe pasar a property obligatoria**.

Modelo correcto para esta pantalla

Ámbito correcto

property

Regla operativa

1 CSV = 1 propiedad

Qué debe hacer el usuario

1. seleccionar propiedad
 2. elegir perfil
 3. subir CSV
 4. dry-run
 5. commit
 6. ver runs/imports de esa propiedad
-

Qué hay que cambiar en frontend

1. ScreeningCsvPage debe dejar de depender solo de orgId

Debe recibir o resolver también la propiedad activa.

Opción buena

```
type Props = {  
  orgId: string;  
  propertyId: string | null;  
};
```

Si no quieres pasarlo por props, puedes leerlo del selector global como haces en Revenue, pero estructuralmente mejor que entre como prop desde la shell.

2. Debe haber selector visible de propiedad en esta pantalla

Aunque exista uno global en cabecera, aquí yo lo mostraría también o al menos reflejaría claramente cuál está activa.

Porque esta pantalla crea datos sensibles. No conviene que el usuario suba un CSV sin ver claramente a qué hotel lo está imputando.

Comportamiento

- si no hay propertyId seleccionado → bloquear acciones
 - deshabilitar:
 - Ver runs
 - Ver importaciones
 - Nuevo screening
 - mostrar aviso:
 - “Selecciona una propiedad para continuar”
-

3. useScreeningRuns debe filtrar por propiedad

Ahora tienes:

```
const { runs } = useScreeningRuns(orgId, 1);
```

Debe pasar a algo así:

```
const { runs } = useScreeningRuns({  
  orgId,  
  propertyId,  
  limit: 1,  
});
```

Y lo mismo en `RunsList`.

4. `ImportWizard` debe trabajar con `propertyId`

Ahora el wizard solo usa:

- `orgId`
- `profileId`
- `runType`
- `filePath`

Eso no vale ya.

Debe incluir `propertyId` en:

- `upload metadata` si aplica
- `dry-run`
- `commit`

Props nuevas

```
type Props = {
  open: boolean;
  orgId: string;
  propertyId: string | null;
  onClose: () => void;
  onCommitted: (runId: string) => void;
};
```

Validación

`canDryRun` debe exigir también `propertyId`.

5. `ImportJobsDialog` debe filtrar por propiedad

Ahora lista importaciones por org.

Eso para auditoría global puede existir, pero esta pantalla operativa no debería abrir todo mezclado.

Debe ser:

```
useImportJobs({
  orgId,
  propertyId,
  from,
  to,
  status,
  runType,
  limit: 200,
})
```

6. RunDetail debería mostrar la propiedad del run

Aquí hay dos posibilidades:

Mejor opción

El backend del detalle devuelve también:

- `property_id`
- `property_name`

Y arriba del run muestras algo así:

- Run: FUTURE_BOOKINGS
- Propiedad: HOTEL_4

Eso evita errores humanos.

Qué cambia en backend

Aquí está lo importante de verdad.

Todas las Edge Functions de este flujo deben aceptar `property_id`

Afectadas seguro

- `upload/import validate/commit`
 - `list runs`
 - `run detail`
 - `list import jobs`
-

Validación obligatoria en backend

En cada función:

1. comprobar `org_id`
 2. comprobar `property_id`
 3. validar que esa `property_id` pertenece a la `org_id`
 4. rechazar si no cuadra
-

Qué tablas deben llevar `property_id`

En este flujo, como mínimo, estas familias deben llevarlo:

Obligatorio

- `screening_runs`
- `import_jobs`
- `import_rows` si aplica como tabla de staging por job
- `screening_results`
- `screening_alerts` si están ligadas al run/resultados

Recomendado también

- cualquier tabla intermedia de staging o snapshot
 - audit event de creación del run/job
-

Cómo quedaría cada componente

ScreeningCsvPage

Debe pasar de:

```
type Props = { orgId: string };  
a:
```

```
type Props = {  
  orgId: string;  
  propertyId: string | null;  
  propertyName?: string | null;  
};
```

Y `canUse` debe ser algo así:

```
const canUse = useMemo(() => {  
  return String(orgId || "").trim().length > 0 && String(propertyId ||  
    "").trim().length > 0;  
}, [orgId, propertyId]);
```

Y si no hay propiedad:

```
<Alert severity="warning">  
  Selecciona una propiedad para subir un CSV y ejecutar screening.  
</Alert>
```

RunsList

Debe recibir `propertyId`:

```
type Props = {  
  orgId: string;  
  propertyId: string;  
  selectedRunId: string | null;  
  onSelectRun: (runId: string) => void;  
  onCreateNew: () => void;  
};
```

ImportWizard

Debe recibir `propertyId` y enviarlo en `dry-run/commit`.

ImportJobsDialog

Debe recibir `propertyId`.

RunDetail

Idealmente no necesita `orgId` si `runId` ya identifica todo, pero eso depende del backend. Si lo mantienes, mejor que el backend confirme también que el run pertenece a:

- esa org
 - esa property
-

Decisión funcional importante

Los runs deben ser de propiedad, no de org

Eso te simplifica todo después.

Un run debe quedar conceptualmente así:

- org: grupo propietario
- property: hotel concreto
- run_type
- created_at
- totals
- alerts

No un run “global del grupo”.

Lo que yo haría ya en esta pantalla

Cambios mínimos imprescindibles

1. añadir `propertyId` a props
2. bloquear pantalla si no hay propiedad seleccionada
3. pasar `propertyId` a:
 - `useScreeningRuns`
 - `ImportWizard`
 - `ImportJobsDialog`
4. hacer que el backend filtre runs/jobs por `property_id`

Mejora recomendable

5. mostrar la propiedad activa en cabecera de la pantalla
 6. mostrar la propiedad en el detalle del run
-

Riesgo si no lo haces

Si mantienes esta pantalla a nivel org mientras Revenue ya vive por propiedad, el producto queda partido:

- revenue bien modelado por hotel
- screening mal imputado a nivel grupo

Y entonces no podrás cruzar bien:

- riesgo del huésped
 - hotel donde ocurre
 - impacto revenue por propiedad
 - alertas y reporting
-

Qué necesito para cerrarla del todo

Ahora ya me has pasado el front principal. Para cerrarla completa, el siguiente paso es que me pegues estas piezas:

1. `useScreeningRuns`
2. `useRunDetail`

3. useImportJobs
4. useImportProfiles
5. screeningCsv.service.ts
6. las Edge Functions que usan estas llamadas

Con eso te lo dejo ya resuelto pantalla por pantalla, incluyendo:

- qué props cambian
- qué payload cambia
- qué tabla tiene que llevar `property_id`
- y si quieres, el diff exacto de frontend para esta pantalla.

```
ESTE // src/hooks/useScreeningRuns.ts import { useCallback, useEffect, useMemo, useState } from
"react"; import type { ScreeningRun } from "@types/screeningCsv.types"; import
{ listScreeningRuns } from "@services/screeningCsv.service"; type State = { loading: boolean;
error: string | null; runs: ScreeningRun[]; }; export function useScreeningRuns(orgId: string, limit =
50) { const [state, setState] = useState<State>({ loading: false, error: null, runs: [], }); const
canLoad = useMemo(() => String(orgId || "").trim().length > 0, [orgId]); const reload =
useCallback(async () => { if (!canLoad) { setState((s) => ({ ...s, loading: false, error: null, runs:
[] })); return; } setState((s) => ({ ...s, loading: true, error: null })); try { const runs = await
listScreeningRuns(orgId, limit); setState({ loading: false, error: null, runs }); } catch (e: any)
{ setState({ loading: false, error: String(e?.message || e), runs: [] }); } }, [orgId, limit, canLoad]);
useEffect(() => { reload(); }, [reload]); return { ...state, reload, }; } // src/hooks/useRunDetail.ts
import { useCallback, useEffect, useMemo, useState } from "react"; import type { ScreeningRun,
ScreeningResult, ScreeningAlert, } from "@types/screeningCsv.types"; import { getScreeningRun,
listRunResults, listRunAlerts, } from "@services/screeningCsv.service"; type Filters =
{ riskBand?: string; onlyChanged?: boolean; unresolvedAlertsOnly?: boolean; }; type State =
{ loading: boolean; error: string | null; run: ScreeningRun | null; results: ScreeningResult[]; alerts:
ScreeningAlert[]; }; export function useRunDetail(runId: string | null, filters?: Filters) { const [state,
setState] = useState<State>({ loading: false, error: null, run: null, results: [], alerts: [], }); const
canLoad = useMemo(() => { return String(runId || "").trim().length > 0; }, [runId]); const reload =
useCallback(async () => { if (!canLoad || !runId) { setState((s) => ({ ...s, loading: false, error: null,
run: null, results: [], alerts: [], })); return; } setState((s) => ({ ...s, loading: true, error: null })); try
{ const [run, results, alerts] = await Promise.all([ getScreeningRun(runId), listRunResults({ runId,
limit: 500, riskBand: filters?.riskBand, onlyChanged: filters?.onlyChanged, }),
listRunAlerts({ runId, limit: 300, unresolvedOnly: filters?.unresolvedAlertsOnly, }), ]);
setState({ loading: false, error: null, run, results, alerts, }); } catch (e: any) { setState({ loading:
false, error: String(e?.message || e), run: null, results: [], alerts: [], }); } }, [runId, canLoad,
filters?.riskBand, filters?.onlyChanged, filters?.unresolvedAlertsOnly]); useEffect(() =>
{ reload(); }, [reload]); return { ...state, reload, }; } SIGUINETE // src/hooks/useImportProfiles.ts
import { useCallback, useEffect, useMemo, useState } from "react"; import type { ImportProfile }
from "@types/screeningCsv.types"; import { listImportProfiles } from
"@services/screeningCsv.service"; type State = { loading: boolean; error: string | null; profiles:
ImportProfile[]; }; export function useImportProfiles(orgId: string) { const [state, setState] =
```

```

useState<State>({ loading: false, error: null, profiles: [], }); const canLoad = useMemo(() =>
String(orgId || "").trim().length > 0, [orgId]); const reload = useCallback(async () => { if (!canLoad)
{ setState((s) => ({ ...s, loading: false, error: null, profiles: [] })); return; } setState((s) => ({ ...s,
loading: true, error: null })); try { const profiles = await listImportProfiles(orgId);
setState({ loading: false, error: null, profiles }); } catch (e: any) { setState({ loading: false, error:
String(e?.message || e), profiles: [] }); } }, [orgId, canLoad]); useEffect(() => { // auto-load reload();
}, [reload]); return { ...state, reload, }; } SIGUINTE // src/hooks/useImportJobs.ts import
{ useCallback, useEffect, useMemo, useState } from "react"; import { supabase } from
"@/services/supabaseClient"; export type ImportJobRow = { id: string; created_at: string | null;
run_type: string | null; status: string | null; file_path: string | null; total_rows: number | null;
valid_rows: number | null; invalid_rows: number | null; summary: any | null; }; export type
ImportJobsFilters = { from?: string; // ISO (inclusive) to?: string; // ISO (inclusive; usamos lte)
status?: string; runType?: string; limit?: number; }; export function useImportJobs(orgId: string,
filters: ImportJobsFilters) { const [loading, setLoading] = useState(false); const [error, setError] =
useState<string | null>(null); const [jobs, setJobs] = useState<ImportJobRow[]>([]); const
stableFilters = useMemo(() => { return { from: filters?.from, to: filters?.to, status: filters?.status,
runType: filters?.runType, limit: filters?.limit ?? 200, }; }, [filters?.from, filters?.to, filters?.status,
filters?.runType, filters?.limit]); const load = useCallback(async () => { const oid = String(orgId ||
 "").trim(); if (!oid) { setJobs([]); setError(null); setLoading(false); return; } setLoading(true);
setError(null); try { // ↻ Cast para saltar el typing cuando faltan tablas/columnas en Database
types let q = (supabase as any) .from("import_jobs") .select("id, created_at, run_type, status,
file_path, total_rows, valid_rows, invalid_rows, summary") .eq("org_id", oid) .order("created_at", {
ascending: false }) .limit(stableFilters.limit); if (stableFilters.from) q = q.gte("created_at",
stableFilters.from); if (stableFilters.to) q = q.lte("created_at", stableFilters.to); if (stableFilters.status
&& stableFilters.status !== "ALL") { q = q.eq("status", stableFilters.status); } if
(stableFilters.runType && stableFilters.runType !== "ALL") { q = q.eq("run_type",
stableFilters.runType); } const { data, error } = await q; if (error) throw error; setJobs((data || []) as
ImportJobRow[]); } catch (e: any) { setError(String(e?.message || e)); setJobs([]); } finally
{ setLoading(false); } }, [orgId, stableFilters]); useEffect(() => { load(); }, [load]); return { loading,
error, jobs, reload: load }; } Y TAMBIEN // src/services/screeningCsv.service.ts import
{ supabase } from "@/services/supabaseClient"; import type { ImportProfile, ScreeningRun,
ScreeningResult, ScreeningAlert, ImportValidateCommitDryRunResponse,
ImportValidateCommitCommitResponse, } from "@/types/screeningCsv.types"; /** * Bucket
donde se suben CSVs. * Default: customer-exports */ const IMPORT_BUCKET =
import.meta.env.VITE_DEBACU_IMPORT_BUCKET || "customer-imports"; /** * ⚠
IMPORTANTE: * Si tu "Database" tipado no incluye estas tablas
(import_profiles/screening_runs/...), * TypeScript subraya .from(...) en rojo. * Solución rápida:
usar un client "any" solo en este servicio. */ const sb: any = supabase; function mustOrgId(orgId:
string) { const v = String(orgId || "").trim(); if (!v) throw new Error("missing_org_id"); return v; }
function pickErrorMessage(e: any, fallback = "request_failed") { return e?.message ||
e?.error_description || e?.error?.message || e?.error || fallback; } // ----- // Profiles
// ----- export async function listImportProfiles(orgId: string):
Promise<ImportProfile[]> { const org_id = mustOrgId(orgId); const { data, error } = await
sb .from("import_profiles") .select("*") .eq("org_id", org_id) .order("updated_at", { ascending:
false }); if (error) throw new Error(pickErrorMessage(error)); return (data || []) as ImportProfile[]; }
// ----- // Runs // ----- export async function

```

```

listScreeningRuns(orgId: string, limit = 50): Promise<ScreeningRun[]> { const org_id =
mustOrgId(orgId); const { data, error } = await sb .from("screening_runs") .select("*") .eq("org_id",
org_id) .order("created_at", { ascending: false }) .limit(limit); if (error) throw new
Error(pickErrorMessage(error)); return (data || []) as ScreeningRun[]; } export async function
getScreeningRun(runId: string): Promise<ScreeningRun | null> { const id = String(runId ||
"").trim(); if (!id) return null; const { data, error } = await
sb .from("screening_runs") .select("*") .eq("id", id) .maybeSingle(); if (error) throw new
Error(pickErrorMessage(error)); return (data || null) as ScreeningRun | null; } //
----- // Results + Alerts // ----- export async function
listRunResults(params: { runId: string; limit?: number; offset?: number; riskBand?: string; //
"HIGH" | "MEDIUM" | "LOW" onlyChanged?: boolean; }): Promise<ScreeningResult[]> { const
run_id = String(params.runId || "").trim(); if (!run_id) return []; const limit =
Math.min(Math.max(Number(params.limit ?? 200), 1), 1000); const offset =
Math.max(Number(params.offset ?? 0), 0); let q =
sb .from("screening_results") .select("*") .eq("run_id", run_id) .order("risk_band", { ascending:
true }) .order("delta_total_net_loss", { ascending: false }) .order("delta_incidents_count",
{ ascending: false }) .range(offset, offset + limit - 1); if (params.riskBand) q = q.eq("risk_band",
params.riskBand); if (params.onlyChanged === true) q = q.eq("risk_band_changed", true); const
{ data, error } = await q; if (error) throw new Error(pickErrorMessage(error)); return (data || []) as
ScreeningResult[]; } export async function listRunAlerts(params: { runId: string; limit?: number;
offset?: number; unresolvedOnly?: boolean; }): Promise<ScreeningAlert[]> { const run_id =
String(params.runId || "").trim(); if (!run_id) return []; const limit =
Math.min(Math.max(Number(params.limit ?? 200), 1), 1000); const offset =
Math.max(Number(params.offset ?? 0), 0); let q =
sb .from("screening_alerts") .select("*") .eq("run_id", run_id) .order("created_at", { ascending:
false }) .range(offset, offset + limit - 1); if (params.unresolvedOnly === true) q =
q.is("resolved_at", null); const { data, error } = await q; if (error) throw new
Error(pickErrorMessage(error)); return (data || []) as ScreeningAlert[]; } //
----- // Storage upload helper // ----- export async function
uploadScreeningCsvToStorage(orgId: string, file: File) { const cleanOrgId = String(orgId ||
"").trim(); if (!cleanOrgId) throw new Error("missing_orgId"); // 1) comprobar sesión REAL const {
data: s, error: sErr } = await supabase.auth.getSession(); if (sErr) throw sErr; if (!s.session) throw
new Error("no_session_before_upload"); // 2) path EXACTO const ts = Date.now(); const
safeName = String(file.name || "upload.csv").replace(/[^\w.\-]+/g, "_"); const path =
debacu_eval/org/${cleanOrgId}/screening/${ts}_${safeName}; // 3) upload
const { data, error } = await supabase.storage .from("customer-imports") .upload(path, file,
{ upsert: false, contentType: file.type || "text/csv", }); if (error) { // Esto es CLAVE: no pierdas el
detalle console.error("[uploadScreeningCsvToStorage] error", { path, error }); throw new
Error(error.message); } return data.path; // esto es lo que luego mandas al edge function } //
----- // Edge Function: import_validate_commit // -----
export async function importValidateCommit(params: { orgId: string; profileId: string; runType?:
string; dryRun: boolean; filePath?: string; csvText?: string; }):
Promise<ImportValidateCommitDryRunResponse | ImportValidateCommitCommitResponse>
{ const payload: any = { org_id: params.orgId, profile_id: params.profileId, run_type:
params.runType, dry_run: params.dryRun, }; if (params.filePath) payload.file_path =
params.filePath; if (params.csvText) payload.csv_text = params.csvText; const { data, error } =

```

```
await sb.functions.invoke("import_validate_commit", { body: payload, }); if (error) throw new Error(pickErrorMessage(error)); if (!data?.ok) throw new Error(String(data?.detail || data?.error || "request_failed")); return data as any; }
```

Bien. Con esto ya se ve casi todo el frente de esta pantalla.

La conclusión sigue siendo la misma, pero ahora ya la podemos aterrizar con precisión:

Diagnóstico real de esta primera pantalla

Ahora mismo el flujo está a nivel org

Se ve en todos los puntos clave:

- `useScreeningRuns(orgId, limit)`
- `useImportProfiles(orgId)`
- `useImportJobs(orgId, filters)`
- `listScreeningRuns(orgId, limit)`
- `listImportProfiles(orgId)`
- `uploadScreeningCsvToStorage(orgId, file)`
- `importValidateCommit({ orgId, profileId, ... })`

Y además:

- `getScreeningRun(runId)`
- `listRunResults({ runId })`
- `listRunAlerts({ runId })`

trabajan por `run_id`, pero **no validan contexto de propiedad desde frontend**.

Eso significa que el dato nace ambiguo y luego ya todo cuelga de ese origen ambiguo.

Qué partes están bien y qué partes hay que cambiar

Lo que está bien

- La pantalla está modularizada.
- El wizard, runs y jobs están separados.
- El `runId` como pivote de detalle está bien.
- El flujo `dry-run -> commit -> run detail` está bien.

Lo que hay que cambiar

Hay que meter `propertyId` en **todo el circuito de creación y listado**:

1. selección de pantalla
 2. listado de runs
 3. listado de import jobs
 4. upload
 5. dry-run
 6. commit
 7. idealmente, detalle de run con propiedad visible
-

Decisión cerrada para esta pantalla

Ámbito correcto

`property`

Regla cerrada

1 CSV = 1 propiedad

Consecuencia

Esta pantalla no puede operar solo con `orgId`.
Tiene que exigir también `propertyId`.

Qué cambiaría exactamente

1. `ScreeningCsvPage`

Debe pasar de:

```
type Props = { orgId: string };  
a algo así:
```

```
type Props = {  
  orgId: string;  
  propertyId: string | null;  
  propertyName?: string | null;  
};
```


Nueva regla de uso

```
const canUse = useMemo(() => {
  return (
    String(orgId || "").trim().length > 0 &&
    String(propertyId || "").trim().length > 0
  );
}, [orgId, propertyId]);
```

Y si no hay propiedad seleccionada, mostrar aviso y bloquear acciones.

2. useScreeningRuns

Ahora:

```
export function useScreeningRuns(orgId: string, limit = 50)
```

Debe pasar a:

```
type UseScreeningRunsParams = {
  orgId: string;
  propertyId: string | null;
  limit?: number;
};
export function useScreeningRuns({
  orgId,
  propertyId,
  limit = 50,
}: UseScreeningRunsParams)
```

Y `canLoad` debe exigir ambos:

```
const canLoad = useMemo(() => {
  return (
    String(orgId || "").trim().length > 0 &&
    String(propertyId || "").trim().length > 0
  );
}, [orgId, propertyId]);
```

Y luego:

```
const runs = await listScreeningRuns({
  orgId,
  propertyId: propertyId!,
  limit,
});
```

3. useImportProfiles

Aquí hay una decisión importante.

Opción correcta por modelo

Los `import_profiles` pueden ser:

- **org-level**, si el perfil describe formato de CSV reutilizable por todo el grupo
- **property-level**, si cada hotel puede tener perfiles distintos

Mi recomendación práctica

Por ahora déjalos a **nivel org**, salvo que ya sepas que cada propiedad tendrá mapeos distintos.

Entonces `useImportProfiles(orgId)` puede quedarse así.

No es el punto crítico de esta pantalla.

Lo crítico es el `run/import/job/result`.

4. useImportJobs

Ahora:

```
export function useImportJobs(orgId: string, filters: ImportJobsFilters)
```

Debe pasar a:

```
export function useImportJobs(  
  orgId: string,  
  propertyId: string | null,  
  filters: ImportJobsFilters  
)
```

Y la query:

```
.eq("org_id", oid)  
.eq("property_id", pid)
```

No por org solo.

Si luego quieres una pantalla corporativa de importaciones, haces otra vista global. Pero esta pantalla operativa debe filtrar por propiedad.

5. screeningCsv.service.ts

Aquí está el corazón del cambio.

listScreeningRuns

Ahora:

```
export async function listScreeningRuns(orgId: string, limit = 50)
```

Debe pasar a:

```
export async function listScreeningRuns(params: {  
  orgId: string;  
  propertyId: string;  
  limit?: number;  
}): Promise<ScreeningRun[]>
```

y la query:

```
.eq("org_id", org_id)  
.eq("property_id", property_id)
```

uploadScreeningCsvToStorage

Ahora sube a:

```
debacu_eval/org/${cleanOrgId}/screening/${ts}_${safeName}
```

Esto se queda corto.

Debe meter también propiedad en el path:

```
debacu_eval/org/${cleanOrgId}/property/${cleanPropertyId}/screening/${ts}_${safeName}
```

Eso es mejor por:

- trazabilidad
- limpieza
- debug
- mantenimiento
- borrado selectivo futuro

La firma debería pasar a:

```
export async function uploadScreeningCsvToStorage(  
  orgId: string,  
  propertyId: string,  
  file: File  
)
```

importValidateCommit

Ahora manda:

```
{  
  org_id,  
  profile_id,  
  run_type,  
  dry_run,  
  file_path,  
}
```

Debe mandar también:

property_id

O sea:

```
const payload: any = {  
  org_id: params.orgId,  
  property_id: params.propertyId,  
  profile_id: params.profileId,  
  run_type: params.runType,  
  dry_run: params.dryRun,  
};
```

Y su firma:

```
export async function importValidateCommit(params: {  
  orgId: string;  
  propertyId: string;  
  profileId: string;  
  runType?: string;  
  dryRun: boolean;
```

```
    filePath?: string;
    csvText?: string;
  })
```

6. ImportWizard

Debe recibir propertyId.

Firma nueva:

```
type Props = {
  open: boolean;
  orgId: string;
  propertyId: string | null;
  onClose: () => void;
  onCommitted: (runId: string) => void;
};
```

Y canDryRun:

```
const canDryRun = useMemo(() => {
  return (
    String(orgId || "").trim().length > 0 &&
    String(propertyId || "").trim().length > 0 &&
    String(profileId || "").trim().length > 0 &&
    !!file &&
    !busy
  );
}, [orgId, propertyId, profileId, file, busy]);
```

Y al subir:

```
const filePath = await uploadScreeningCsvToStorage(orgId, propertyId!, file);
```

Y en importValidateCommit:

```
const res = await importValidateCommit({
  orgId,
  propertyId: propertyId!,
  profileId: selectedProfile.id,
  runType,
  dryRun: true,
  filePath,
});
```

Lo mismo en commit.

7. ImportJobsDialog

Debe recibir propertyId.

Firma:

```
type Props = {
  open: boolean;
  onClose: () => void;
  orgId: string;
  propertyId: string | null;
};
```

Y al cargar:

```
const { loading, error, jobs, reload } = useImportJobs(orgId, propertyId, { ... });
```

Si no hay propertyId, no debería cargar nada.

8. RunsList

Debe recibir propertyId.

Firma:

```
type Props = {  
  orgId: string;  
  propertyId: string;  
  selectedRunId: string | null;  
  onSelectRun: (runId: string) => void;  
  onCreateNew: () => void;  
};
```

9. RunDetail

Aquí hay una matización.

A corto plazo

Puede seguir trabajando por runId, **si el backend garantiza que ese run ya pertenece a una propiedad concreta.**

Pero idealmente

La tabla screening_runs debería tener:

- property_id
- opcionalmente property_name en join/view

y getScreeningRun(runId) debería devolver también eso para mostrarlo.

Ejemplo arriba del detalle:

- Run: FUTURE_BOOKINGS
- Propiedad: HOTEL_4

Eso es muy recomendable.

Qué cambia en base de datos

Con el código que me has pasado, estas tablas deben revisarse ya:

Obligatorias con `property_id`

- `screening_runs`
- `import_jobs`
- `screening_results`
- `screening_alerts`

Probablemente también

- tablas staging del import
- import rows si existen
- cualquier tabla snapshot derivada

Si alguna de estas no tiene `property_id`, hay que añadirlo.

Qué cambia en backend

Y aquí está la parte crítica.

La Edge Function `import_validate_commit` debe:

1. recibir `property_id`
2. validar que pertenece a `org_id`
3. guardar el `import_job` con `property_id`
4. guardar el `run` con `property_id`
5. guardar resultados/alertas con `property_id` o al menos heredarlo por relación sólida al `run`

Si no hace eso, el frontend nuevo no arregla el problema real.

Lo que ya te puedo decir sin ver la Edge Function

La arquitectura correcta para esta pantalla es esta

`import_profiles`

Puede quedarse org-level por ahora.

import_jobs

Property-level.

screening_runs

Property-level.

screening_results

Property-level.

screening_alerts

Property-level.

Te dejo el cambio conceptual cerrado

Esta pantalla no es

“screening CSV del grupo”

Esta pantalla es

“screening CSV para una propiedad del grupo”

Y eso debe verse en:

- selector
 - upload path
 - payload
 - tablas
 - queries
 - detalle del run
-

Orden exacto de implementación para esta pantalla

Paso 1

Cambiar props de ScreeningCsvPage, ImportWizard, ImportJobsDialog, RunsList.

Paso 2

Cambiar hooks:

- `useScreeningRuns`
- `useImportJobs`

Paso 3

Cambiar servicio:

- `listScreeningRuns`
- `uploadScreeningCsvToStorage`
- `importValidateCommit`

Paso 4

Cambiar Edge Function `import_validate_commit`

Paso 5

Añadir `property_id` a tablas si falta

Paso 6

Mostrar propiedad activa en UI

Qué necesito ahora para dejar esta pantalla cerrada de verdad

Lo siguiente ya no es más frontend. Ahora necesito ver:

1. la Edge Function `import_validate_commit`
2. si tienes función para listar runs/import jobs por RPC o lectura directa
3. estructura de tablas:
 - `screening_runs`
 - `import_jobs`
 - `screening_results`
 - `screening_alerts`

Con eso ya te digo exactamente:

- qué SQL tocar

- qué payload tocar
- y en qué orden migrarlo sin romper lo que ya tienes.

FUNCION:

```
// supabase/functions/import_validate_commit/index.ts
// deno-lint-ignore-file no-explicit-any
import "jsr:@supabase/functions-js/edge-runtime.d.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.45.4";
import { json, preflight } from "../_shared/cors.ts";
import { requireUser } from "../_shared/auth.ts";

function mustEnv(name: string) {
  const v = Deno.env.get(name);
  if (!v) throw new Error(`MISSING_ENV:${name}`);
  return v;
}

const SUPABASE_URL = mustEnv("SUPABASE_URL");
const SERVICE_ROLE_KEY = mustEnv("SUPABASE_SERVICE_ROLE_KEY");
const GLOBAL_PEPPER = mustEnv("DEBACU_GLOBAL_PEPPER");

// ☒ Storage bucket donde guardas los CSV
const IMPORT_BUCKET = Deno.env.get("DEBACU_IMPORT_BUCKET") || "customer-imports";

// -----
// Supabase admin client (Service Role)
// -----
function sbAdmin() {
  return createClient(SUPABASE_URL, SERVICE_ROLE_KEY, {
    auth: { persistSession: false, autoRefreshToken: false },
  });
}

// -----
// Utils
// -----
function str(v: unknown) {
  return String(v ?? "").trim();
}

function lower(v: unknown) {
  return str(v).toLowerCase();
}

function upper(v: unknown) {
  return str(v).toUpperCase();
}

function num(v: unknown): number | null {
  if (v === null || v === undefined || v === "") return null;
  const n = Number(String(v).replace(",", "."));
  return Number.isFinite(n) ? n : null;
}

function toISODate(v: unknown): string | null {
  const t = str(v);
  if (!t) return null;
  const m = t.match(/^(?(\d{4})-(\d{2})-(\d{2}))/);
  return m ? m[1] : null;
}

function digitsOnly(v: unknown) {
  const d = str(v).replace(/\D/g, "");
}
```

```

    return d || "";
}

function normalizeEmail(v: unknown) {
    const e = lower(v);
    if (!e || !e.includes("@")) return null;
    return e;
}

function normalizeDocument(v: unknown) {
    const t = str(v);
    if (!t) return null;
    const out = t.replace(/[^A-Za-z0-9]/g, "").toUpperCase();
    return out || null;
}

function normalizePhone(v: unknown) {
    const d = digitsOnly(v);
    if (!d || d.length < 7) return null;
    return d;
}

function chunk<T>(arr: T[], size: number) {
    const out: T[][] = [];
    for (let i = 0; i < arr.length; i += size) out.push(arr.slice(i, i + size));
    return out;
}

function todayIsoTs() {
    return new Date().toISOString();
}

function daysBetweenFromToday(dateISO: string) {
    const d = new Date(dateISO);
    const ms = Date.now() - d.getTime();
    if (!Number.isFinite(ms)) return null;
    return Math.max(0, Math.floor(ms / 86400000));
}

function minDate(a: string | null, b: string | null): string | null {
    if (!a) return b;
    if (!b) return a;
    return a <= b ? a : b;
}

function maxDate(a: string | null, b: string | null): string | null {
    if (!a) return b;
    if (!b) return a;
    return a >= b ? a : b;
}

// -----
// ☒ IMPORT TYPE mapping (para pasar debacu eval import_batches_type_chk)
// -----
function mapImportType(runType: string) {
    const rt = String(runType || "").toUpperCase();

    // Tu CHECK actual permite: FUTURE_BOOKINGS, PAST_STAYS, GUEST_STAYS, INCIDENTS
    if (rt === "FUTURE_BOOKINGS") return "FUTURE_BOOKINGS";
    if (rt === "HISTORICAL_STAYS") return "PAST_STAYS";
    if (rt === "HISTORICAL_BOOKINGS") return "PAST_STAYS";
    if (rt === "INHOUSE_TODAY") return "GUEST_STAYS";

    // fallback seguro:

```

```

if (rt.includes("FUTURE")) return "FUTURE_BOOKINGS";
if (rt.includes("PAST") || rt.includes("HISTORICAL")) return "PAST_STAYS";
if (rt.includes("GUEST") || rt.includes("INHOUSE")) return "GUEST_STAYS";
if (rt.includes("INCIDENT")) return "INCIDENTS";

return "GUEST_STAYS";
}

// -----
// ☒ Guard: membership check (ACTIVE) para org_id
// -----
async function requireOrgMemberActive(
  admin: ReturnType<typeof sbAdmin>,
  org_id: string,
  user_id: string,
) {
  const { data, error } = await admin
    .from("debacu_eval_org_members")
    .select("id, status, role")
    .eq("org_id", org_id)
    .eq("user_id", user_id)
    .maybeSingle();

  if (error) throw new Error(`org_member_check_failed:${error.message}`);
  if (!data?.id) return { ok: false as const, reason: "NOT_MEMBER" as const };
  if (String(data.status || "").toUpperCase() !== "ACTIVE") {
    return { ok: false as const, reason: "NOT_ACTIVE" as const };
  }

  return { ok: true as const, member: data };
}

// -----
// CSV parser (simple, soporta comillas y comas)
// -----
function parseCsv(csvText: string, delimiter = ",") {
  const rows: string[][] = [];
  let cur = "";
  let inQuotes = false;
  let row: string[] = [];

  for (let i = 0; i < csvText.length; i++) {
    const ch = csvText[i];

    if (ch === '"') {
      const next = csvText[i + 1];
      if (inQuotes && next === '"') {
        cur += '"';
        i++;
      } else {
        inQuotes = !inQuotes;
      }
      continue;
    }
  }
}

```

```

    if (!inQuotes && ch === delimiter) {
      row.push(cur);
      cur = "";
      continue;
    }

    if (!inQuotes && (ch === "\n" || ch === "\r")) {
      if (ch === "\r" && csvText[i + 1] === "\n") i++;
      row.push(cur);
      cur = "";
      const allEmpty = row.every((c) => str(c) === "");
      if (!allEmpty) rows.push(row);
      row = [];
      continue;
    }

    cur += ch;
  }

  if (cur.length || row.length) {
    row.push(cur);
    const allEmpty = row.every((c) => str(c) === "");
    if (!allEmpty) rows.push(row);
  }

  return rows;
}

// -----
// Identity HMAC
// -----
async function hmacHex(message: string) {
  const enc = new TextEncoder();
  const key = await crypto.subtle.importKey(
    "raw",
    enc.encode(GLOBAL_PEPPER),
    { name: "HMAC", hash: "SHA-256" },
    false,
    ["sign"],
  );
  const sig = await crypto.subtle.sign("HMAC", key, enc.encode(message));
  return Array.from(new Uint8Array(sig))
    .map((b) => b.toString(16).padStart(2, "0"))
    .join("");
}

type IdentityInfo = {
  identity_key: string | null;
  match_basis: "DOCUMENT" | "EMAIL" | "PHONE" | "NAME_DOB" | null;
  match_confidence: "HIGH" | "MEDIUM" | "LOW";
  document_norm: string | null;
  email_norm: string | null;
  phone_digits: string | null;
};

```

```

doc_key: string | null;
email_key: string | null;
phone_key: string | null;
};

```

```

async function computeIdentity(row: any, strategy: string): Promise<IdentityInfo> {
  const doc = normalizeDocument(row.document_number ?? row.documentNumber);
  const docCountry = upper(row.document_country ?? row.documentCountry);
  const email = normalizeEmail(row.email);
  const phone = normalizePhone(row.phone);
  const dob = toISODate(row.date_of_birth ?? row.dateOfBirth);
  const first = str(row.first_name ?? row.firstName);
  const last = str(row.last_name ?? row.lastName);

```

```

const st = String(strategy || "").toUpperCase();

```

```

let raw: string | null = null;
let basis: IdentityInfo["match_basis"] = null;
let conf: IdentityInfo["match_confidence"] = "LOW";

```

```

if (st === "DOCUMENT_STRONG") {
  if (doc && docCountry) {
    raw = `DOC:${doc}|C:${docCountry}`;
    basis = "DOCUMENT";
    conf = "HIGH";
  } else if (doc) {
    raw = `DOC:${doc}`;
    basis = "DOCUMENT";
    conf = "HIGH";
  }
} else if (st === "DOCUMENT") {
  if (doc) {
    raw = `DOC:${doc}`;
    basis = "DOCUMENT";
    conf = "HIGH";
  }
} else if (st === "EMAIL") {
  if (email) {
    raw = `EMAIL:${email}`;
    basis = "EMAIL";
    conf = "MEDIUM";
  }
} else if (st === "PHONE") {
  if (phone) {
    raw = `PHONE:${phone}`;
    basis = "PHONE";
    conf = "MEDIUM";
  }
} else if (st === "NAME_DOB") {
  if (first && last && dob) {
    raw = `NAME_DOB:${upper(first)}|${upper(last)}|${dob}`;
    basis = "NAME_DOB";
  }
}

```

```
    conf = "LOW";
  }
}
```

```
if (!raw) {
  if (doc) {
    raw = `DOC:${doc}`;
    basis = "DOCUMENT";
    conf = "HIGH";
  } else if (email) {
    raw = `EMAIL:${email}`;
    basis = "EMAIL";
    conf = "MEDIUM";
  } else if (phone) {
    raw = `PHONE:${phone}`;
    basis = "PHONE";
    conf = "MEDIUM";
  } else if (first && last && dob) {
    raw = `NAME_DOB:${upper(first)}|${upper(last)}|${dob}`;
    basis = "NAME_DOB";
    conf = "LOW";
  }
}
```

```
const doc_key = doc ? await hmacHex(`DOC:${doc}`) : null;
const email_key = email ? await hmacHex(`EMAIL:${email}`) : null;
const phone_key = phone ? await hmacHex(`PHONE:${phone}`) : null;
```

```
if (!raw) {
  return {
    identity_key: null,
    match_basis: null,
    match_confidence: "LOW",
    document_norm: doc,
    email_norm: email,
    phone_digits: phone,
    doc_key,
    email_key,
    phone_key,
  };
}
```

```
const identity_key = await hmacHex(raw);
return {
  identity_key,
  match_basis: basis,
  match_confidence: conf,
  document_norm: doc,
  email_norm: email,
  phone_digits: phone,
  doc_key,
  email_key,
```

```

    phone_key,
  };
}

```

```

// -----
// Required fields por run_type
// -----
function requiredFieldsFor(runType: string) {
  const rt = String(runType || "").toUpperCase();

  if (rt === "INHOUSE_TODAY") return ["checkin_date", "first_name", "last_name"];
  if (rt === "FUTURE_BOOKINGS") return ["checkin_date", "checkout_date", "first_name", "last_name"];
  if (rt === "HISTORICAL_STAYS") return ["checkin_date", "checkout_date", "status", "channel",
"first_name", "last_name"];
  if (rt === "HISTORICAL_BOOKINGS") return ["booking_created_at", "checkin_date", "checkout_date",
"status", "channel", "first_name", "last_name"];
  return ["checkin_date", "first_name", "last_name"];
}

```

```

// -----
// Mapping: hotel_column -> debacu_field
// -----
function applyMapping(headers: string[], values: string[], mapping: Record<string, string>) {
  const obj: Record<string, any> = {};

```

```

  const DIRECT_FIELDS = new Set([
    "checkin_date",
    "checkout_date",
    "booking_created_at",
    "status",
    "channel",
    "currency",
    "first_name",
    "last_name",
    "full_name",
    "email",
    "phone",
    "document_number",
    "document_country",
    "reservation_ref",
    "total_amount",
    "room_amount",
    "extras_amount",
    "commission_amount",
    "net_amount",
    "deposit_amount",
    "date_of_birth",
  ]);

```

```

const ALIASES: Record<string, string> = {
  document: "document_number",
  doc: "document_number",
  dni: "document_number",
  passport: "document_number",

```

```

telefono: "phone",
movil: "phone",
mail: "email",
e_mail: "email",
checkin: "checkin_date",
checkout: "checkout_date",
arrival_date: "checkin_date",
departure_date: "checkout_date",
};

for (let i = 0; i < headers.length; i++) {
  const rawH = String(headers[i] ?? "").trim();
  const v = values[i] ?? "";
  if (!rawH) continue;

  const mapped = mapping?.[rawH];
  if (mapped) {
    obj[mapped] = v;
    continue;
  }

  const h = rawH.toLowerCase();
  const normalized = (ALIASES[h] ?? rawH).trim();
  const normalizedLower = normalized.toLowerCase();

  if (DIRECT_FIELDS.has(normalizedLower)) {
    obj[normalizedLower] = v;
  }
}

return obj;
}

// -----
// Storage read
// -----
async function readCsvFromStorage(admin: ReturnType<typeof sbAdmin>, filePath: string) {
  const { data, error } = await admin.storage.from(IMPORT_BUCKET).download(filePath);
  if (error || !data) throw new Error(`storage_download_failed:${error?.message || "no_data"}`);
  return await data.text();
}

// -----
// Prev run finder (para delta)
// -----
async function getPreviousRunId(
  admin: ReturnType<typeof sbAdmin>,
  org_id: string,
  run_type: string,
  current_run_id: string,
) {
  const { data: cur, error: curErr } = await admin
    .from("screening_runs")
    .select("id, created_at")

```



```

    .eq("id", current_run_id)
    .maybeSingle();
    if (curErr || !cur?.created_at) return null;

```

```

const { data: prev, error: prevErr } = await admin
    .from("screening_runs")
    .select("id")
    .eq("org_id", org_id)
    .eq("run_type", run_type)
    .lt("created_at", cur.created_at)
    .order("created_at", { ascending: false })
    .limit(1)
    .maybeSingle();

```

```

    if (prevErr || !prev?.id) return null;
    return String(prev.id);
}

```

```

// -----
// Risk decay
// -----
function decayFactor(days_since_last: number | null) {
    if (days_since_last === null || days_since_last === undefined) return 1;
    if (days_since_last < 90) return 1;
    if (days_since_last < 180) return 0.8;
    if (days_since_last < 365) return 0.5;
    return 0.25;
}

function bandRank(b: string) {
    return b === "HIGH" ? 3 : b === "MEDIUM" ? 2 : 1;
}

function confRank(c: string) {
    return c === "HIGH" ? 3 : c === "MEDIUM" ? 2 : 1;
}

```

```

// -----
// stay_status mapping (NO NULL)
// -----
function deriveStayStatus(runType: string, csvStatus: string | null) {
    const rt = String(runType || "").toUpperCase();
    const s = upper(csvStatus || "");

```

```

    if (s) {
        if (["PLANNED", "CHECKED_IN", "COMPLETED", "NO_SHOW", "CANCELLED"].includes(s)) return s;

```

```

        if (["BOOKED", "RESERVED", "CONFIRMED", "PENDING"].includes(s)) return "PLANNED";
        if (["INHOUSE", "IN_HOUSE", "CHECKEDIN", "CHECKED-IN"].includes(s)) return "CHECKED_IN";
        if (["CHECKEDOUT", "CHECKED_OUT", "CHECKED-OUT", "DONE", "DEPARTED"].includes(s)) return "COMPLETED";
        if (["CANCELED", "CANCEL", "ANULADA", "ANULADO"].includes(s)) return "CANCELLED";
        if (["NOSHOW", "NO-SHOW"].includes(s)) return "NO_SHOW";

```

```

    }
    return "PLANNED";
}

```

```

    if (rt === "FUTURE_BOOKINGS") return "PLANNED";

```

```

if (rt === "INHOUSE_TODAY") return "CHECKED_IN";
if (rt === "HISTORICAL_STAYS") return "COMPLETED";
if (rt === "HISTORICAL_BOOKINGS") return "PLANNED";

return "PLANNED";
}

// -----
// MAIN
// -----
Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return preflight(req);
  if (req.method !== "POST") return json(req, 405, { ok: false, error: "method_not_allowed" });

  try {
    const user = await requireUser(req);
    const userId = String(user?.id || "");
    if (!userId) return json(req, 401, { ok: false, error: "UNAUTHENTICATED" });

    const body = await req.json().catch(() => null);
    if (!body) return json(req, 400, { ok: false, error: "invalid_json_body" });

    const admin = sbAdmin();

    const org_id = str(body.org_id ?? body.orgId);
    const profile_id = str(body.profile_id ?? body.profileId);
    const run_type = str(body.run_type ?? body.runType);
    const dry_run = body.dry_run === true || body.dryRun === true;
    const file_path = str(body.file_path ?? body.filePath);
    const csv_text = str(body.csv_text ?? body.csvText);

    if (!org_id) return json(req, 400, { ok: false, error: "missing_org_id" });
    if (!profile_id) return json(req, 400, { ok: false, error: "missing_profile_id" });
    if (!file_path && !csv_text) return json(req, 400, { ok: false, error: "missing_file_or_csv_text" });

    // ☒ Security gate
    const mem = await requireOrgMemberActive(admin, org_id, userId);
    if (!mem.ok) {
      return json(req, 403, {
        ok: false,
        error: "FORBIDDEN",
        detail: mem.reason === "NOT_ACTIVE" ? "org_membership_not_active" : "org_membership_missing",
      });
    }
  }

  // 1) Load profile
  const { data: profile, error: pErr } = await admin
    .from("import_profiles")
    .select("id, org_id, source_type, delimiter, identity_strategy, mapping, disabled_fields")
    .eq("id", profile_id)
    .eq("org_id", org_id)
    .maybeSingle();

```

```
if (pErr || !profile) return json(req, 404, { ok: false, error: "profile_not_found" });
```

```
const effectiveRunType = String(run_type || profile.source_type || "").toUpperCase();
const import_type = mapImportType(effectiveRunType);
const delimiter = String(profile.delimiter || ",", "");
const identity_strategy = String(profile.identity_strategy || "DOCUMENT_STRONG");
const mapping = (profile.mapping || {}) as Record<string, string>;
const disabled = new Set<string>((profile.disabled_fields || []) as string[]);
```

```
// 2) Read CSV
```

```
const csv = csv_text ? csv_text : await readCsvFromStorage(admin, file_path);
```

```
// 3) Parse
```

```
const rawRows = parseCsv(csv, delimiter);
```

```
if (rawRows.length < 2) return json(req, 400, { ok: false, error: "csv_empty_or_missing_rows" });
```

```
const headers = rawRows[0].map((h) => str(h));
```

```
const dataRows = rawRows.slice(1);
```

```
const totalRows = dataRows.length;
```

```
// 4) Create import job
```

```
const file_hash = await crypto.subtle
```

```
.digest("SHA-256", new TextEncoder().encode(csv))
```

```
.then((buf) => Array.from(new Uint8Array(buf)).map((b) => b.toString(16).padStart(2, "0")).join(""));
```

```
const { data: job, error: jobErr } = await admin
```

```
.from("import_jobs")
```

```
.insert({
```

```
  org_id,
```

```
  user_id: userId,
```

```
  profile_id,
```

```
  file_path: file_path || "(inline)",
```

```
  file_hash,
```

```
  run_type: effectiveRunType,
```

```
  status: "UPLOADED",
```

```
});
```

```
.select("id")
```

```
.single();
```

```
if (jobErr || !job?.id) {
```

```
  return json(req, 500, { ok: false, error: "import_job_create_failed", detail: jobErr?.message });
```

```
}
```

```
const required = requiredFieldsFor(effectiveRunType).filter((f) => !disabled.has(f));
```

```
// 5) Validate & normalize rows
```

```
const errors: Array<{ row: number; field?: string; error: string }> = [];
```

```
const parsed: any[] = [];
```

```
for (let i = 0; i < dataRows.length; i++) {
```

```
  const rowNum = i + 2;
```

```
const values = dataRows[i];
const obj = applyMapping(headers, values, mapping);
```

```
if (!str(obj.first_name) || !str(obj.last_name)) {
  const fn = str(obj.full_name);
  if (fn) {
    const parts = fn.split(/\s+/).filter(Boolean);
    if (!str(obj.first_name) && parts.length >= 1) obj.first_name = parts[0];
    if (!str(obj.last_name) && parts.length >= 2) obj.last_name = parts.slice(1).join(" ");
  }
}
```

```
obj.checkin_date = toISODate(obj.checkin_date);
obj.checkout_date = toISODate(obj.checkout_date);
obj.booking_created_at = str(obj.booking_created_at || "");
obj.status = upper(obj.status);
obj.channel = str(obj.channel);
obj.currency = upper(obj.currency);
```

```
for (const f of required) {
  const v = obj[f];
  if (!str(v)) errors.push({ row: rowNum, field: f, error: "missing_required_field" });
}
```

```
const ident = await computeIdentity(obj, identity_strategy);
if (!ident.identity_key) errors.push({ row: rowNum, field: "identity", error: "NO_IDENTIFIER" });
```

```
obj.total_amount = num(obj.total_amount);
obj.room_amount = num(obj.room_amount);
obj.extras_amount = num(obj.extras_amount);
obj.commission_amount = num(obj.commission_amount);
obj.net_amount = num(obj.net_amount);
obj.deposit_amount = num(obj.deposit_amount);
```

```
obj.identity_key = ident.identity_key;
obj.match_basis = ident.match_basis;
obj.match_confidence = ident.match_confidence;
obj.document_norm = ident.document_norm;
obj.email_norm = ident.email_norm;
obj.phone_digits = ident.phone_digits;
obj.doc_key = ident.doc_key;
obj.email_key = ident.email_key;
obj.phone_key = ident.phone_key;
```

```
if (!str(obj.full_name)) obj.full_name = `${str(obj.first_name)} ${str(obj.last_name)}`.trim();
```

```
parsed.push(obj);
}
```

```
const invalidRowNums = new Set(errors.map((e) => e.row));
const validRows = parsed.filter((_, idx) => !invalidRowNums.has(idx + 2));
```

```

await admin
  .from("import_jobs")
  .update({
    total_rows: totalRows,
    valid_rows: validRows.length,
    invalid_rows: totalRows - validRows.length,
    status: "VALIDATED",
    summary: {
      run_type: effectiveRunType,
      required_fields: required,
      errors_count: errors.length,
    },
  })
  .eq("id", job.id);

```

```

if (dry_run) {
  return json(req, 200, {
    ok: true,
    mode: "DRY_RUN",
    import_job_id: job.id,
    total_rows: totalRows,
    valid_rows: validRows.length,
    invalid_rows: totalRows - validRows.length,
    errors: errors.slice(0, 300),
    preview: validRows.slice(0, 20),
  });
}

```

```

const nowTs = todayIsoTs();

```

```

// 6) Create import_batch
const { data: batchRow, error: bErr } = await admin
  .from("debacu_eval_import_batches")
  .insert({
    org_id,
    import_type, // ☒ YA MAPEADO -> pasa el check constraint
    total_rows: totalRows,
    processed_rows: totalRows,
    error_rows: 0,
    file_name: file_path || null,
    created_at: nowTs,
  })
  .select("id")
  .single();

```

```

if (bErr || !batchRow?.id) {
  await admin.from("import_jobs").update({ status: "FAILED" }).eq("id", job.id);
  return json(req, 500, { ok: false, error: "import_batch_create_failed", detail: bErr?.message });
}

```

```

const importBatchId = String(batchRow.id);

```

```
// 7) Create screening_run
const { data: run, error: runErr } = await admin
  .from("screening_runs")
  .insert({
    org_id,
    import_job_id: job.id,
    run_type: effectiveRunType,
    total_analyzed: 0,
    high_count: 0,
    medium_count: 0,
    low_count: 0,
  })
  .select("id")
  .single();
```

```
if (runErr || !run?.id) {
  await admin.from("import_jobs").update({ status: "FAILED" }).eq("id", job.id);
  return json(req, 500, { ok: false, error: "screening_run_create_failed", detail: runErr?.message });
}
```

```
const runId = String(run.id);
const prevRunId = await getPreviousRunId(admin, org_id, effectiveRunType, runId);
```

```
// Prev snapshot map
const prevMap = new Map<string, { incidents: number; loss: number; band: string }>();
if (prevRunId) {
  const { data: prevRows, error: prevErr } = await admin
    .from("screening_results")
    .select("identity_key,checkin_date,incidents_count,total_net_loss,risk_band")
    .eq("run_id", prevRunId);
```

```
if (!prevErr && prevRows) {
  for (const pr of prevRows as any[]) {
    const ik = String(pr.identity_key || "");
    const cd = pr.checkin_date ? String(pr.checkin_date) : "";
    if (!ik || !cd) continue;
    prevMap.set(`${ik}|${cd}`, {
      incidents: Number(pr.incidents_count ?? 0),
      loss: Number(pr.total_net_loss ?? 0),
      band: String(pr.risk_band ?? "LOW"),
    });
  }
}
```

```
// Preload guest_index
const uniqueIdentityKeys = Array.from(new Set(validRows.map((r) => String(r.identity_key || ""))).filter(Boolean));
```

```
type GuestIndexRow = {
  identity_key: string;
  stays_count: number;
```

```

    incidents_count: number;
    total_net_loss: number;
    first_seen_date: string | null;
    last_seen_date: string | null;
    last_incident_date: string | null;
    risk_band: string | null;
    doc_key: string | null;
    email_key: string | null;
    phone_key: string | null;
};

```

```

const indexMap = new Map<string, GuestIndexRow>();

```

```

for (const part of chunk(uniqueIdentityKeys, 1000)) {
    const { data: idxRows, error: idxErr } = await admin
        .from("debacu_eval_guest_index")

```

```

        .select("identity_key,stays_count,incidents_count,total_net_loss,first_seen_date,last_seen_date,last_incident_date,risk_band,doc_key,email_key,phone_key")
        .in("identity_key", part);

```

```

    if (idxErr) {
        await admin.from("import_jobs").update({ status: "FAILED" }).eq("id", job.id);
        return json(req, 500, { ok: false, error: "guest_index_preload_failed", detail: idxErr.message });
    }

```

```

(idxRows || []).forEach((r: any) => {
    indexMap.set(String(r.identity_key), {
        identity_key: String(r.identity_key),
        stays_count: Number(r.stays_count ?? 0),
        incidents_count: Number(r.incidents_count ?? 0),
        total_net_loss: Number(r.total_net_loss ?? 0),
        first_seen_date: r.first_seen_date ? String(r.first_seen_date) : null,
        last_seen_date: r.last_seen_date ? String(r.last_seen_date) : null,
        last_incident_date: r.last_incident_date ? String(r.last_incident_date) : null,
        risk_band: r.risk_band ? String(r.risk_band) : null,
        doc_key: r.doc_key ? String(r.doc_key) : null,
        email_key: r.email_key ? String(r.email_key) : null,
        phone_key: r.phone_key ? String(r.phone_key) : null,
    });
});

```

```

for (const ik of part) {
    if (!indexMap.has(ik)) {
        indexMap.set(ik, {
            identity_key: ik,
            stays_count: 0,
            incidents_count: 0,
            total_net_loss: 0,
            first_seen_date: null,
            last_seen_date: null,
            last_incident_date: null,
            risk_band: null,

```

```

        doc_key: null,
        email_key: null,
        phone_key: null,
    });
}
}
}
}

```

```

// Consolidation
type Agg = {
    identity_key: string;
    checkin_date: string;
    row_number: number;
    checkout_date: string | null;
    match_confidence: string;
    match_basis: string | null;
    incidents_count: number;
    total_net_loss: number;
    last_incident_date: string | null;
    days_since_last: number | null;
    risk_band: string;
    prev_risk_band: string | null;
    risk_band_changed: boolean;
    delta_incidents_count: number;
    delta_total_net_loss: number;
    full_name: string;
    stay_status: string;
    doc_key: string | null;
    email_key: string | null;
    phone_key: string | null;
};

```

```

const agg = new Map<string, Agg>();
const RESULTS_CHUNK = 1000;
const ALERTS_CHUNK = 1000;
const STAYS_CHUNK = 1000;
const INDEX_CHUNK = 1000;

```

```

for (let i = 0; i < validRows.length; i++) {
    const r = validRows[i];

```

```

    const identity_key = String(r.identity_key);
    const checkin_date = (r.checkin_date as string | null) || null;
    const checkout_date = (r.checkout_date as string | null) || null;
    if (!checkin_date) continue;

```

```

    const row_number = i + 2;

```

```

    const idxRow = indexMap.get(identity_key);
    const incidents_count = Number(idxRow?.incidents_count ?? 0);
    const total_net_loss = Number(idxRow?.total_net_loss ?? 0);
    const base_band = String(idxRow?.risk_band ?? "LOW");

```



```
const last_incident_date = idxRow?.last_incident_date ?? null;
```

```
const days_since_last = last_incident_date ? daysBetweenFromToday(last_incident_date) : null;  
const df = decayFactor(days_since_last);  
const adjusted_loss = total_net_loss * df;
```

```
let risk_band = base_band;  
if (incidents_count === 0 && adjusted_loss <= 0) risk_band = "LOW";  
else if (incidents_count >= 2 || adjusted_loss >= 200) risk_band = "HIGH";  
else risk_band = "MEDIUM";
```

```
const key = `${identity_key}|${checkin_date}`;
```

```
let prev_band: string | null = null;  
let risk_band_changed = false;  
let delta_incidents = incidents_count;  
let delta_loss = total_net_loss;
```

```
if (prevMap.has(key)) {  
  const prev = prevMap.get(key)!;  
  prev_band = prev.band;  
  risk_band_changed = prev.band !== risk_band;  
  delta_incidents = incidents_count - prev.incidents;  
  delta_loss = total_net_loss - prev.loss;  
}
```

```
if (!Number.isFinite(delta_incidents)) delta_incidents = 0;  
if (!Number.isFinite(delta_loss)) delta_loss = 0;
```

```
const full_name = str(r.full_name) || `${str(r.first_name)} ${str(r.last_name)}.trim();  
const stay_status = deriveStayStatus(effectiveRunType, r.status || null);
```

```
const doc_key = r.doc_key ? String(r.doc_key) : null;  
const email_key = r.email_key ? String(r.email_key) : null;  
const phone_key = r.phone_key ? String(r.phone_key) : null;
```

```
const existing = agg.get(key);  
if (!existing) {  
  agg.set(key, {  
    identity_key,  
    checkin_date,  
    row_number,  
    checkout_date,  
    match_confidence: r.match_confidence || "LOW",  
    match_basis: r.match_basis || null,  
    incidents_count,  
    total_net_loss,  
    last_incident_date,  
    days_since_last,  
    risk_band,  
    prev_risk_band: prev_band,  
    risk_band_changed,
```

```

    delta_incidents_count: delta_incidents,
    delta_total_net_loss: delta_loss,
    full_name,
    stay_status,
    doc_key,
    email_key,
    phone_key,
  });
} else {
  if (row_number < existing.row_number) existing.row_number = row_number;
  if (checkout_date && (!existing.checkout_date || checkout_date > existing.checkout_date))
existing.checkout_date = checkout_date;

```

```

const mc = String(r.match_confidence || "LOW");
if (confRank(mc) > confRank(existing.match_confidence)) {
  existing.match_confidence = mc;
  existing.match_basis = r.match_basis || null;
}

```

```

if (bandRank(risk_band) > bandRank(existing.risk_band)) {
  existing.risk_band = risk_band;
  existing.risk_band_changed = prev_band ? prev_band !== risk_band : false;
  existing.prev_risk_band = prev_band;
}

```

```

if (delta_incidents > existing.delta_incidents_count) existing.delta_incidents_count =
delta_incidents;
if (delta_loss > existing.delta_total_net_loss) existing.delta_total_net_loss = delta_loss;

```

```

if (!existing.doc_key && doc_key) existing.doc_key = doc_key;
if (!existing.email_key && email_key) existing.email_key = email_key;
if (!existing.phone_key && phone_key) existing.phone_key = phone_key;

```

```

const prio = (x: string) => (x === "CHECKED_IN" ? 5 : x === "PLANNED" ? 4 : x === "COMPLETED" ? 3 :
x === "NO_SHOW" ? 2 : 1);
if (prio(stay_status) > prio(existing.stay_status)) existing.stay_status = stay_status;
}
}

```

```

const consolidated = Array.from(agg.values());

```

```

let high = 0, med = 0, lowc = 0;
for (const a of consolidated) {
  if (a.risk_band === "HIGH") high++;
  else if (a.risk_band === "MEDIUM") med++;
  else lowc++;
}

```

```

// -----
// ☒ UPSERT STAYS (NO DUPLICADOS)
// Requiere UNIQUE INDEX: (org_id, identity_key, checkin_date)
// -----
for (const part of chunk(consolidated, STAYS_CHUNK)) {

```

```
const batch = part.map((a) => ({
  org_id,
  identity_key: a.identity_key,
  full_name: a.full_name || null,
  checkin_date: a.checkin_date,
  checkout_date: a.checkout_date,
  stay_status: a.stay_status,
  import_batch_id: importBatchId,
  updated_at: nowTs, // ☒ tu tabla tiene updated_at
}));
```

```
const { error: sErr } = await admin
  .from("debacu_eval_guest_stays")
  .upsert(batch, { onConflict: "org_id,identity_key,checkin_date" });
```

```
if (sErr) {
  await admin.from("import_jobs").update({ status: "FAILED" }).eq("id", job.id);
  return json(req, 500, { ok: false, error: "guest_stays_upsert_failed", detail: sErr.message });
}
}
```

```
// -----
// ☒ UPSERT guest_index (stays_count + seen_dates)
// ⚠ Nota: si quieres que NO sume stays_count al reimportar el mismo CSV,
// hay que calcular "stays_added" = filas realmente nuevas. Esto es otro paso.
// -----
type StayAgg = { stays_added: number; min_ci: string | null; max_co: string | null; doc_key: string |
null; email_key: string | null; phone_key: string | null };
const perIdentity = new Map<string, StayAgg>();
```

```
for (const a of consolidated) {
  const ik = a.identity_key;
  const cur = perIdentity.get(ik) || { stays_added: 0, min_ci: null, max_co: null, doc_key: null,
email_key: null, phone_key: null };
  cur.stays_added += 1;
  cur.min_ci = minDate(cur.min_ci, a.checkin_date);
  cur.max_co = maxDate(cur.max_co, a.checkout_date || a.checkin_date);
  if (!cur.doc_key && a.doc_key) cur.doc_key = a.doc_key;
  if (!cur.email_key && a.email_key) cur.email_key = a.email_key;
  if (!cur.phone_key && a.phone_key) cur.phone_key = a.phone_key;
  perIdentity.set(ik, cur);
}
```

```
const upserts: any[] = [];
for (const [ik, sa] of perIdentity.entries()) {
  const existing = indexMap.get(ik) || {
    identity_key: ik,
    stays_count: 0,
    incidents_count: 0,
    total_net_loss: 0,
    first_seen_date: null,
    last_seen_date: null,
    last_incident_date: null,
```

```

    risk_band: null,
    doc_key: null,
    email_key: null,
    phone_key: null,
  };

```

```

const next_first = minDate(existing.first_seen_date, sa.min_ci);
const next_last = maxDate(existing.last_seen_date, sa.max_co);

```

```

upserts.push({
  identity_key: ik,
  stays_count: Number(existing.stays_count ?? 0) + Number(sa.stays_added ?? 0),
  first_seen_date: next_first,
  last_seen_date: next_last,
  incidents_count: Number(existing.incidents_count ?? 0),
  total_net_loss: Number(existing.total_net_loss ?? 0),
  last_incident_date: existing.last_incident_date,
  risk_band: existing.risk_band || "LOW",
  doc_key: existing.doc_key || sa.doc_key,
  email_key: existing.email_key || sa.email_key,
  phone_key: existing.phone_key || sa.phone_key,
  updated_at: nowTs,
});
}

```

```

for (const part of chunk(upserts, INDEX_CHUNK)) {
  const { error: uErr } = await admin
    .from("debacu_eval_guest_index")
    .upsert(part, { onConflict: "identity_key" });
}

```

```

if (uErr) {
  await admin.from("import_jobs").update({ status: "FAILED" }).eq("id", job.id);
  return json(req, 500, { ok: false, error: "guest_index_upsert_failed", detail: uErr.message });
}
}

```

```

// screening_results
for (const part of chunk(consolidated, RESULTS_CHUNK)) {
  const batch = part.map((a) => ({
    run_id: runId,
    org_id,
    identity_key: a.identity_key,
    row_number: a.row_number,
    checkin_date: a.checkin_date,
    risk_band: a.risk_band,
    prev_risk_band: a.prev_risk_band,
    risk_band_changed: a.risk_band_changed,
    incidents_count: a.incidents_count,
    total_net_loss: a.total_net_loss,
    last_incident_date: a.last_incident_date,
    days_since_last: a.days_since_last,
    delta_incidents_count: a.delta_incidents_count,
  }));
}

```

```

    delta_total_net_loss: a.delta_total_net_loss,
    match_confidence: a.match_confidence,
    match_basis: a.match_basis,
    computed_at: nowTs,
  }));

```

```

const { error: insErr } = await admin.from("screening_results").insert(batch);
if (insErr) {
  await admin.from("import_jobs").update({ status: "FAILED" }).eq("id", job.id);
  return json(req, 500, { ok: false, error: "screening_results_insert_failed", detail: insErr.message
});
}
}
}

```

```

// alerts
const alertsAll = consolidated
  .filter((a) => a.delta_incidents_count > 0 || a.delta_total_net_loss > 0 || a.risk_band_changed ||
a.risk_band === "HIGH")
  .map((a) => {
    const alert_type =
      a.risk_band === "HIGH"
        ? "HIGH_RISK"
        : a.risk_band_changed
        ? "RISK_CHANGED"
        : a.delta_incidents_count > 0
        ? "NEW_INCIDENT"
        : "DELTA_CHANGE";

```

```

    return {
      org_id,
      run_id: runId,
      identity_key: a.identity_key,
      row_number: a.row_number,
      alert_type,
      message: `Update detected (band=${a.risk_band}, Δinc=${a.delta_incidents_count}, Δloss=${
a.delta_total_net_loss})`,
    };
  });
}

```

```

for (const part of chunk(alertsAll, ALERTS_CHUNK)) {
  const { error: aErr } = await admin.from("screening_alerts").insert(part);
  if (aErr) {
    await admin.from("import_jobs").update({ status: "FAILED" }).eq("id", job.id);
    return json(req, 500, { ok: false, error: "screening_alerts_insert_failed", detail:
aErr.message });
  }
}
}

```

```

await admin
  .from("screening_runs")
  .update({
    total_analyzed: consolidated.length,
    high_count: high,

```

```

        medium_count: med,
        low_count: lowc,
    })
    .eq("id", runId);

```

```

    await admin
    .from("import_jobs")
    .update({
        status: "COMMITTED",
        summary: {
            run_id: runId,
            prev_run_id: prevRunId,
            total_rows: totalRows,
            valid_rows: validRows.length,
            invalid_rows: totalRows - validRows.length,
            consolidated_rows: consolidated.length,
            high,
            medium: med,
            low: lowc,
            import_batch_id: importBatchId,
            import_type,
        },
    })
    .eq("id", job.id);

```

```

    return json(req, 200, {
        ok: true,
        mode: "COMMIT",
        import_job_id: job.id,
        import_batch_id: importBatchId,
        run_id: runId,
        prev_run_id: prevRunId,
        total_rows: totalRows,
        valid_rows: validRows.length,
        invalid_rows: totalRows - validRows.length,
        consolidated_rows: consolidated.length,
        guest_index_upserts: upserts.length,
        high,
        medium: med,
        low: lowc,
        errors: errors.slice(0, 300),
    });
} catch (e: any) {
    console.error("import_validate_commit error:", e);
    const msg = String(e?.message || e);

```

```

    if (msg.startsWith("MISSING_ENV:")) return json(req, 500, { ok: false, error: "missing_env", detail:
msg });
    if (msg === "UNAUTHENTICATED" || msg === "UNAUTHORIZED") return json(req, 401, { ok: false, error:
"UNAUTHENTICATED" });

```

```

    return json(req, 500, { ok: false, error: "request_failed", detail: msg });
}

```

)):

estructura de tablas:

- screening_runs
- import_jobs
- screening_results
- screening_alerts

Aquí ya sí se ve el punto crítico real.

Y además confirms algo importante: **no usáis RPC**, así que el ajuste correcto es 100% en **Edge Functions + tablas + frontend**, que es como debe ser en vuestro modelo.

Diagnóstico exacto de esta Edge Function

Ahora mismo `import_validate_commit` está bien resuelta para org, pero **no está resuelta para propiedad**.

Qué hace bien

- valida usuario autenticado
- valida membresía activa en `debacu_eval_org_members`
- carga `import_profile` por `org_id`
- procesa CSV
- crea `import_job`
- crea `debacu_eval_import_batches`
- crea `screening_run`
- inserta `screening_results`
- inserta `screening_alerts`
- actualiza `guest_stays`
- actualiza `guest_index`

Qué le falta

`property_id` no entra en ningún punto del flujo.

Eso significa que todo esto queda imputado solo a org:

- `import_jobs`
- `debacu_eval_import_batches`
- `screening_runs`
- `screening_results`

- `screening_alerts`
- `debacu_eval_guest_stays` si quieres trazabilidad por hotel
- posiblemente hasta parte del contexto futuro de IA y reporting

Y eso es justo el problema que habías detectado.

Decisión cerrada para esta función

Esta Edge Function debe pasar a trabajar con:

- `org_id`
- `property_id`

Regla de validación

Debe comprobar:

1. el usuario pertenece a la org
2. la propiedad pertenece a la org
3. si todo cuadra, procesa

Cambios exactos que haría en `import_validate_commit`

1. Leer `property_id` del body

Ahora tienes:

```
const org_id = str(body.org_id ?? body.orgId);
const profile_id = str(body.profile_id ?? body.profileId);
const run_type = str(body.run_type ?? body.runType);
```

Debes añadir:

```
const property_id = str(body.property_id ?? body.propertyId);
```

Y validar:

```
if (!property_id) return json(req, 400, { ok: false, error:
"missing_property_id" });
```

2. Crear guard de propiedad activa dentro de org

Necesitas una función parecida a `requireOrgMemberActive`, pero para propiedad.

Algo así:

```
async function requirePropertyInOrg(
  admin: ReturnType<typeof sbAdmin>,
  org_id: string,
  property_id: string,
) {
  const { data, error } = await admin
    .from("debacu_eval_properties")
    .select("id, org_id, name, status")
    .eq("id", property_id)
    .eq("org_id", org_id)
    .maybeSingle();
  if (error) throw new Error(`property_check_failed:${error.message}`);
  if (!data?.id) return { ok: false as const, reason: "PROPERTY_NOT_IN_ORG" as const };
  const status = String(data.status || "").toUpperCase();
  if (status && status !== "ACTIVE") {
    return { ok: false as const, reason: "PROPERTY_NOT_ACTIVE" as const };
  }
  return { ok: true as const, property: data };
}
```

Y luego en el main:

```
const prop = await requirePropertyInOrg(admin, org_id, property_id);
if (!prop.ok) {
  return json(req, 403, {
    ok: false,
    error: "FORBIDDEN",
    detail:
      prop.reason === "PROPERTY_NOT_ACTIVE"
        ? "property_not_active"
        : "property_not_in_org",
  });
}
```

3. Guardar property_id en import_jobs

Ahora insertas:

```
.insert({
  org_id,
  user_id: userId,
  profile_id,
  file_path: file_path || "(inline)",
  file_hash,
  run_type: effectiveRunType,
  status: "UPLOADED",
})
```

Debe pasar a:

```
.insert({
  org_id,
  property_id,
  user_id: userId,
  profile_id,
  file_path: file_path || "(inline)",
})
```

```
file_hash,  
run_type: effectiveRunType,  
status: "UPLOADED",  
})
```

4. Guardar property_id en debacu_eval_import_batches

Ahora insertas:

```
.insert({  
  org_id,  
  import_type,  
  total_rows: totalRows,  
  processed_rows: totalRows,  
  error_rows: 0,  
  file_name: file_path || null,  
  created_at: nowTs,  
})
```

Debe pasar a:

```
.insert({  
  org_id,  
  property_id,  
  import_type,  
  total_rows: totalRows,  
  processed_rows: totalRows,  
  error_rows: 0,  
  file_name: file_path || null,  
  created_at: nowTs,  
})
```

Si esa tabla aún no tiene property_id, hay que añadirlo.

5. Guardar property_id en screening_runs

Ahora:

```
.insert({  
  org_id,  
  import_job_id: job.id,  
  run_type: effectiveRunType,  
  total_analyzed: 0,  
  high_count: 0,  
  medium_count: 0,  
  low_count: 0,  
})
```

Debe pasar a:

```
.insert({  
  org_id,  
  property_id,  
  import_job_id: job.id,  
  run_type: effectiveRunType,  
  total_analyzed: 0,  
  high_count: 0,  
  medium_count: 0,  
  low_count: 0,  
})
```

```
} )
```

6. getPreviousRunId debe ser por propiedad, no por org global

Este es un punto importante y fácil de pasar por alto.

Ahora buscas el run previo así:

```
.eq("org_id", org_id)
.eq("run_type", run_type)
```

Eso está mal para multi-property, porque cogería el último run del grupo aunque fuera de otro hotel.

Debe pasar a:

```
async function getPreviousRunId(
  admin: ReturnType<typeof sbAdmin>,
  org_id: string,
  property_id: string,
  run_type: string,
  current_run_id: string,
)
```

y dentro:

```
.eq("org_id", org_id)
.eq("property_id", property_id)
.eq("run_type", run_type)
```

Y la llamada:

```
const prevRunId = await getPreviousRunId(admin, org_id, property_id,
effectiveRunType, runId);
```

Esto es crítico. Si no, los deltas quedan contaminados entre hoteles del mismo grupo.

7. Guardar property_id en screening_results

Ahora insertas:

```
{
  run_id: runId,
  org_id,
  identity_key: a.identity_key,
  ...
}
```

Debe pasar a:

```
{
  run_id: runId,
  org_id,
  property_id,
  identity_key: a.identity_key,
  ...
}
```

Aunque ya puedas inferir la propiedad por el run, yo la guardaría también.

Te simplifica:

- filtros

- índices
 - reporting
 - depuración
 - joins menos costosos
-

8. Guardar property_id en screening_alerts

Ahora insertas:

```
{
  org_id,
  run_id: runId,
  identity_key: a.identity_key,
  ...
}
```

Debe pasar a:

```
{
  org_id,
  property_id,
  run_id: runId,
  identity_key: a.identity_key,
  ...
}
```

9. Decisión sobre debacu_eval_guest_stays

Aquí hay matiz.

Ahora haces upsert con:

```
onConflict: "org_id,identity_key,checkin_date"
```

Eso, en multi-property, probablemente ya no vale.

Problema

Un mismo huésped puede tener estancia el mismo día en propiedades distintas del grupo en datos cargados o históricos. No es lo normal, pero puede pasar. Y además conceptualmente una estancia pertenece a una propiedad concreta.

Mi recomendación

debacu_eval_guest_stays debe llevar property_id y el conflicto debería pasar a:

```
onConflict: "org_id,property_id,identity_key,checkin_date"
```

Y el insert:

```
{
  org_id,
  property_id,
  identity_key: a.identity_key,
  ...
}
```

Esto lo cerraría así.

10. Decisión sobre `debacu_eval_guest_index`

Aquí no haría el mismo cambio.

Mi criterio

`guest_index` puede seguir siendo **org-level** si lo que representa es el historial consolidado del huésped dentro del grupo.

Eso tiene bastante sentido para screening global del cliente.

O sea:

- `guest_stays` → property-level
- `guest_index` → org-level consolidado

Eso es coherente.

Pero

Si quieres análisis futuros por propiedad, luego puedes crear:

- o vistas por propiedad
 - o una tabla agregada `property_guest_index`
 - pero no es necesario ahora
-

11. El path de storage debe incluir propiedad

Ahora:

```
const path = `debacu_eval/org/${cleanOrgId}/screening/${ts}_${safeName}`;
```

Debe pasar a algo así:

```
const cleanPropertyId = String(propertyId || "").trim();
if (!cleanPropertyId) throw new Error("missing_propertyId");
const path =
  `debacu_eval/org/${cleanOrgId}/property/${cleanPropertyId}/screening/${ts}_${safeName}`;
```

Y por tanto la firma cambia a:

```
export async function uploadScreeningCsvToStorage(orgId: string, propertyId:
string, file: File)
```

SQL / modelo que casi seguro necesitas

Sin ver las tablas, te diría que casi seguro hay que revisar estas:

Añadir `property_id`

- `import_jobs`
- `debacu_eval_import_batches`
- `screening_runs`
- `screening_results`
- `screening_alerts`
- `debacu_eval_guest_stays`

Probablemente no hace falta añadirlo por ahora

- `debacu_eval_guest_index`
-

Parte muy importante: los listados del frontend ahora están mal para multi-property

Porque aunque arregles esta Edge Function, sigues teniendo lecturas directas por Supabase desde frontend en:

- `listScreeningRuns`
- `getScreeningRun`
- `listRunResults`
- `listRunAlerts`
- `useImportJobs`
- `listImportProfiles`

Y tú mismo has dicho:

todo debe ir por Edge Functions.

Entonces la corrección no es solo meter `property_id`; también es alinear este flujo con vuestra regla arquitectónica.

Lo correcto según vuestro modelo

En vez de lecturas directas desde frontend

Deberíais tener Edge Functions tipo:

- `screening_runs_list`

- `screening_run_detail`
- `screening_import_jobs_list`
- `screening_import_profiles_list`

Y esas funciones deben aceptar:

- `org_id`
- `property_id` cuando aplique
- filtros

Así:

- centralizas seguridad
 - no dependes de RLS compleja en cliente
 - mantienes coherencia con el resto del proyecto
-

Mi recomendación realista para no abrir veinte frentes

Fase mínima para cerrar esta pantalla

1. meter `property_id` en `import_validate_commit`
2. guardar `property_id` en tablas críticas
3. filtrar el run previo por `property_id`
4. cambiar frontend para exigir propiedad
5. cambiar upload path incluyendo propiedad

Fase buena

6. dejar de leer estas tablas directas desde frontend
 7. mover listados/detalle a Edge Functions también
-

Decisiones cerradas para esta pantalla

Decisión 1

Consulta automática CSV es **property-level obligatoria**.

Decisión 2

`import_validate_commit` debe recibir y validar `property_id`.

Decisión 3

`screening_runs`, `import_jobs`, `screening_results`, `screening_alerts` y `guest_stays` deben quedar imputadas a `property_id`.

Decisión 4

El cálculo de `prevRunId` debe hacerse por `org_id + property_id + run_type`.

Decisión 5

`guest_index` puede seguir siendo org-level consolidado.

Decisión 6

Los listados de esta pantalla deberían acabar migrando también a Edge Functions, porque esa es vuestra arquitectura.

Lo siguiente que te pediría para dejar esta primera pantalla cerrada del todo

Pásame ahora, en este orden:

1. esquema o columnas de:
 - `import_jobs`
 - `screening_runs`
 - `screening_results`
 - `screening_alerts`
 - `debacu_eval_import_batches`
 - `debacu_eval_guest_stays`
2. si existe ya:
 - tabla `debacu_eval_properties`
 - columnas básicas

Con eso te doy exactamente:

- qué columnas añadir

- qué índices crear
- qué `onConflict` cambiar
- y luego te devuelvo el **diff completo de frontend + Edge Function** para esta pantalla.

este bloque y sacas todo de una vez en Supabase SQL Editor.

```
select
  table_name,
  column_name,
  data_type,
  is_nullable,
  column_default
from information_schema.columns
where table_schema = 'public'
  and table_name in (
    'import_jobs',
    'screening_runs',
    'screening_results',
    'screening_alerts',
    'debacu_eval_import_batches',
    'debacu_eval_guest_stays',
    'debacu_eval_properties'
  )
order by
  table_name,
  ordinal_position;
```

Si además quieres ver rápido qué tablas de esas existen realmente, lanza también esta:

```
select
  table_name
from information_schema.tables
where table_schema = 'public'
  and table_name in (
    'import_jobs',
    'screening_runs',
    'screening_results',
    'screening_alerts',
    'debacu_eval_import_batches',
    'debacu_eval_guest_stays',
    'debacu_eval_properties'
  )
order by table_name;
```

Y si quieres dejar ya medio cerrado el análisis, saca también constraints e índices de esas mismas tablas:

```
select
  tc.table_name,
  tc.constraint_name,
  tc.constraint_type,
  kcu.column_name
from information_schema.table_constraints tc
left join information_schema.key_column_usage kcu
  on tc.constraint_name = kcu.constraint_name
  and tc.table_schema = kcu.table_schema
  and tc.table_name = kcu.table_name
where tc.table_schema = 'public'
  and tc.table_name in (
    'import_jobs',
    'screening_runs',
    'screening_results',
    'screening_alerts',
```

```

        'debacu_eval_import_batches',
        'debacu_eval_guest_stays',
        'debacu_eval_properties'
    )
order by tc.table_name, tc.constraint_type, tc.constraint_name,
kcu.ordinal_position;
select
    schemaname,
    tablename,
    indexname,
    indexdef
from pg_indexes
where schemaname = 'public'
    and tablename in (
        'import_jobs',
        'screening_runs',
        'screening_results',
        'screening_alerts',
        'debacu_eval_import_batches',
        'debacu_eval_guest_stays',
        'debacu_eval_properties'
    )
order by tablename, indexname;

```

table_name	column_name	data_type	is_nullable	column_default
debacu_eval_guest_stays	id	uuid	NO	gen_random_uuid()
debacu_eval_guest_stays	org_id	uuid	NO	null
debacu_eval_guest_stays	identity_key	text	NO	null
debacu_eval_guest_stays	full_name	text	YES	null
debacu_eval_guest_stays	checkin_date	date	NO	null
debacu_eval_guest_stays	checkout_date	date	YES	null
debacu_eval_guest_stays	stay_status	text	NO	'PLANNED':text
debacu_eval_guest_stays	import_batch_id	uuid	YES	null
debacu_eval_guest_stays	created_at	timestamp with time zone	NO	now()
debacu_eval_guest_stays	updated_at	timestamp with time zone	NO	now()
debacu_eval_import_batches	id	uuid	NO	gen_random_uuid()
debacu_eval_import_batches	org_id	uuid	NO	null
debacu_eval_import_batches	file_name	text	YES	null
debacu_eval_import_batches	import_type	text	NO	null
debacu_eval_import_batches	total_rows	integer	NO	0
debacu_eval_import_batches	processed_rows	integer	NO	0
debacu_eval_import_batches	error_rows	integer	NO	0
debacu_eval_import_batches	created_at	timestamp with time zone	NO	now()
debacu_eval_properties	id	uuid	NO	gen_random_uuid()
debacu_eval_properties	org_id	uuid	NO	null
debacu_eval_properties	code	text	NO	null
debacu_eval_properties	name	text	NO	null

debacu_eval_properties	category	smallint	YES	null	
debacu_eval_properties	address	text	YES	null	
debacu_eval_properties	city	text	YES	null	
debacu_eval_properties	country	text	YES	'ES'::text	
debacu_eval_properties	timezone	text	YES	'Europe/Madrid'::text	
debacu_eval_properties	is_active	boolean	NO	true	
debacu_eval_properties	created_at	timestamp with time zone	NO	now()	
debacu_eval_properties	updated_at	timestamp with time zone	NO	now()	
debacu_eval_properties	legal_name	text	YES	null	
debacu_eval_properties	tax_id	text	YES	null	
debacu_eval_properties	property_type	text	YES	'HOTEL'::text	
debacu_eval_properties	region	text	YES	null	
debacu_eval_properties	currency_code	text	YES	'EUR'::text	
debacu_eval_properties	rooms_total	integer	YES	null	
debacu_eval_properties	created_by	uuid	YES	null	
debacu_eval_properties	import_property_code	text	YES	null	
import_jobs	id	uuid	NO	gen_random_uuid()	
import_jobs	org_id	uuid	NO	null	
import_jobs	user_id	uuid	NO	null	
import_jobs	profile_id	uuid	YES	null	
import_jobs	file_path	text	NO	null	
import_jobs	file_hash	text	YES	null	
import_jobs	run_type	text	NO	null	
import_jobs	total_rows	integer	YES	0	
import_jobs	valid_rows	integer	YES	0	
import_jobs	invalid_rows	integer	YES	0	
import_jobs	status	text	NO	'UPLOADED'::text	
import_jobs	summary	jsonb	YES	null	
import_jobs	created_at	timestamp with time zone	YES	now()	
import_jobs	updated_at	timestamp with time zone	YES	now()	
import_jobs	run_id	uuid	YES	null	
screening_alerts	id	uuid	NO	gen_random_uuid()	
screening_alerts	org_id	uuid	NO	null	
screening_alerts	run_id	uuid	NO	null	
screening_alerts	identity_key	text	NO	null	
screening_alerts	alert_type	text	NO	null	
screening_alerts	message	text	YES	null	
screening_alerts	created_at	timestamp with time zone	YES	now()	
screening_alerts	resolved_at	timestamp with time zone	YES	null	
screening_alerts	severity	text	YES	null	
screening_alerts	code	text	YES	null	
screening_alerts	row_number	integer	YES	null	
screening_alerts	meta	jsonb	YES	null	
screening_results	id	uuid	NO	gen_random_uuid()	
screening_results	run_id	uuid	NO	null	
screening_results	org_id	uuid	NO	null	
screening_results	identity_key	text	NO	null	
screening_results	checkin_date	date	YES	null	
screening_results	risk_band	text	NO	null	
screening_results	incidents_count	integer	YES	null	

screening_results	total_net_loss	numeric	YES	null	
screening_results	last_incident_date	date	YES	null	
screening_results	days_since_last	integer	YES	null	
screening_results	delta_incidents_count	integer	YES	0	
screening_results	delta_total_net_loss	numeric	YES	0	
screening_results	match_confidence	text	YES	null	
screening_results	match_basis	text	YES	null	
screening_results	computed_at	timestamp with time zone	YES	now()	
screening_results	prev_risk_band	text	YES	null	
screening_results	risk_band_changed	boolean	YES	false	
screening_results	row_number	integer	YES	null	
screening_runs	id	uuid	NO	gen_random_uuid()	
screening_runs	org_id	uuid	NO	null	
screening_runs	import_job_id	uuid	YES	null	
screening_runs	run_type	text	NO	null	
screening_runs	total_analyzed	integer	YES	0	
screening_runs	high_count	integer	YES	0	
screening_runs	medium_count	integer	YES	0	
screening_runs	low_count	integer	YES	0	
screening_runs	created_at	timestamp with time zone	YES	now()	
screening_runs	status	text	NO	'DONE'::text	
screening_runs	started_at	timestamp with time zone	YES	null	
screening_runs	finished_at	timestamp with time zone	YES	null	
screening_runs	created_by	uuid	YES	null	
screening_runs	source_ref	text	YES	null	
screening_runs	params	jsonb	YES	null	

table_name	

debacu_eval_guest_stays	
debacu_eval_import_batches	
debacu_eval_properties	
import_jobs	
screening_alerts	
screening_results	
screening_runs	

table_name	constraint_name	constraint_type	column_name
-----	-----	-----	
debacu_eval_guest_stays	2200_78236_10_not_null	CHECK	null
debacu_eval_guest_stays	2200_78236_1_not_null	CHECK	null

debacu_eval_guest_stays	2200_78236_2_not_null	CHECK	null
debacu_eval_guest_stays	2200_78236_3_not_null	CHECK	null
debacu_eval_guest_stays	2200_78236_5_not_null	CHECK	null
debacu_eval_guest_stays	2200_78236_7_not_null	CHECK	null
debacu_eval_guest_stays	2200_78236_9_not_null	CHECK	null
debacu_eval_guest_stays null	debacu_eval_guest_stays_status_chk	CHECK	
debacu_eval_guest_stays KEY import_batch_id	debacu_eval_guest_stays_import_batch_id_fkey	FOREIGN	
debacu_eval_guest_stays id	debacu_eval_guest_stays_pkey	PRIMARY KEY	
debacu_eval_import_batches null	2200_78203_1_not_null	CHECK	
debacu_eval_import_batches null	2200_78203_2_not_null	CHECK	
debacu_eval_import_batches null	2200_78203_4_not_null	CHECK	
debacu_eval_import_batches null	2200_78203_5_not_null	CHECK	
debacu_eval_import_batches null	2200_78203_6_not_null	CHECK	
debacu_eval_import_batches null	2200_78203_7_not_null	CHECK	
debacu_eval_import_batches null	2200_78203_8_not_null	CHECK	
debacu_eval_import_batches null	debacu_eval_import_batches_type_chk	CHECK	
debacu_eval_import_batches KEY id	debacu_eval_import_batches_pkey	PRIMARY	
debacu_eval_properties	2200_84257_10_not_null	CHECK	null
debacu_eval_properties	2200_84257_11_not_null	CHECK	null
debacu_eval_properties	2200_84257_12_not_null	CHECK	null
debacu_eval_properties	2200_84257_1_not_null	CHECK	null
debacu_eval_properties	2200_84257_2_not_null	CHECK	null
debacu_eval_properties	2200_84257_3_not_null	CHECK	null
debacu_eval_properties	2200_84257_4_not_null	CHECK	null
debacu_eval_properties null	debacu_eval_properties_currency_code_chk	CHECK	
debacu_eval_properties CHECK null	debacu_eval_properties_import_property_code_not_blank_chk		

debacu_eval_properties null	debacu_eval_properties_property_type_chk	CHECK	
debacu_eval_properties null	debacu_eval_properties_rooms_total_chk	CHECK	
debacu_eval_properties KEY created_by	debacu_eval_properties_created_by_fkey	FOREIGN	
debacu_eval_properties org_id	debacu_eval_properties_org_id_fkey	FOREIGN KEY	
debacu_eval_properties id	debacu_eval_properties_pkey	PRIMARY KEY	
debacu_eval_properties org_id	debacu_eval_properties_org_id_code_key	UNIQUE	
debacu_eval_properties code	debacu_eval_properties_org_id_code_key	UNIQUE	
import_jobs	2200_79483_11_not_null	CHECK	null
import_jobs	2200_79483_1_not_null	CHECK	null
import_jobs	2200_79483_2_not_null	CHECK	null
import_jobs	2200_79483_3_not_null	CHECK	null
import_jobs	2200_79483_5_not_null	CHECK	null
import_jobs	2200_79483_7_not_null	CHECK	null
import_jobs	import_jobs_run_type_check	CHECK	null
import_jobs	import_jobs_status_check	CHECK	null
import_jobs profile_id	import_jobs_profile_id_fkey	FOREIGN KEY	
import_jobs	import_jobs_pkey	PRIMARY KEY	id
screening_alerts	2200_79564_1_not_null	CHECK	null
screening_alerts	2200_79564_2_not_null	CHECK	null
screening_alerts	2200_79564_3_not_null	CHECK	null
screening_alerts	2200_79564_4_not_null	CHECK	null
screening_alerts	2200_79564_5_not_null	CHECK	null
screening_alerts run_id	screening_alerts_run_id_fkey	FOREIGN KEY	
screening_alerts	screening_alerts_pkey	PRIMARY KEY	id
screening_results	2200_79541_1_not_null	CHECK	null
screening_results	2200_79541_2_not_null	CHECK	null

screening_results	2200_79541_3_not_null	CHECK	null
screening_results	2200_79541_4_not_null	CHECK	null
screening_results	2200_79541_6_not_null	CHECK	null
screening_results null	screening_results_match_confidence_check	CHECK	
screening_results run_id	screening_results_run_id_fkey	FOREIGN KEY	
screening_results	screening_results_pkey	PRIMARY KEY	id
screening_runs	2200_79521_10_not_null	CHECK	null
screening_runs	2200_79521_1_not_null	CHECK	null
screening_runs	2200_79521_2_not_null	CHECK	null
screening_runs	2200_79521_4_not_null	CHECK	null
screening_runs	screening_runs_run_type_check	CHECK	null
screening_runs import_job_id	screening_runs_import_job_id_fkey	FOREIGN KEY	
screening_runs	screening_runs_pkey	PRIMARY KEY	id

schemaname	tablename	indexname	indexdef
-----	-----	-----	

```

-----
| public | debacu_eval_guest_stays | debacu_eval_guest_stays_pkey | CREATE
UNIQUE INDEX debacu_eval_guest_stays_pkey ON public.debacu_eval_guest_stays USING
btree (id)
| public | debacu_eval_guest_stays | debacu_eval_guest_stays_uniq | CREATE
UNIQUE INDEX debacu_eval_guest_stays_uniq ON public.debacu_eval_guest_stays USING
btree (org_id, identity_key, checkin_date)
| public | debacu_eval_guest_stays | debacu_eval_guest_stays_unique_idx | CREATE
UNIQUE INDEX debacu_eval_guest_stays_unique_idx ON public.debacu_eval_guest_stays
USING btree (org_id, identity_key, checkin_date)
| public | debacu_eval_guest_stays | debacu_eval_guest_stays_unique_stay | CREATE
UNIQUE INDEX debacu_eval_guest_stays_unique_stay ON public.debacu_eval_guest_stays
USING btree (org_id, identity_key, checkin_date, checkout_date)
| public | debacu_eval_guest_stays | idx_stays_org_identity | CREATE
INDEX idx_stays_org_identity ON public.debacu_eval_guest_stays USING btree (org_id,
identity_key)
| public | debacu_eval_import_batches | debacu_eval_import_batches_org_idx |
CREATE INDEX debacu_eval_import_batches_org_idx ON public.debacu_eval_import_batches
USING btree (org_id, created_at DESC)
| public | debacu_eval_import_batches | debacu_eval_import_batches_pkey |
CREATE UNIQUE INDEX debacu_eval_import_batches_pkey ON

```

```

public.debacu_eval_import_batches USING btree (id)
|
| public | debacu_eval_properties | debacu_eval_properties_org_code_ux | CREATE
UNIQUE INDEX debacu_eval_properties_org_code_ux ON public.debacu_eval_properties USING
btree (org_id, code)
|
| public | debacu_eval_properties | debacu_eval_properties_org_id_code_key |
CREATE UNIQUE INDEX debacu_eval_properties_org_id_code_key ON
public.debacu_eval_properties USING btree (org_id, code)
|
| public | debacu_eval_properties | debacu_eval_properties_org_id_idx | CREATE
INDEX debacu_eval_properties_org_id_idx ON public.debacu_eval_properties USING btree
(org_id)
|
| public | debacu_eval_properties | debacu_eval_properties_org_import_property_code_idx |
CREATE INDEX debacu_eval_properties_org_import_property_code_idx ON
public.debacu_eval_properties USING btree (org_id, import_property_code) WHERE
(import_property_code IS NOT NULL)
|
| public | debacu_eval_properties | debacu_eval_properties_pkey | CREATE
UNIQUE INDEX debacu_eval_properties_pkey ON public.debacu_eval_properties USING btree
(id)
|
| public | debacu_eval_properties | idx_debacu_eval_properties_org_id | CREATE
INDEX idx_debacu_eval_properties_org_id ON public.debacu_eval_properties USING btree
(org_id)
|
| public | debacu_eval_properties | idx_debacu_eval_properties_org_is_active |
CREATE INDEX idx_debacu_eval_properties_org_is_active ON public.debacu_eval_properties
USING btree (org_id, is_active)
|
| public | debacu_eval_properties | ix_properties_org | CREATE INDEX
ix_properties_org ON public.debacu_eval_properties USING btree (org_id)
|
| public | import_jobs | import_jobs_org_created_at_idx | CREATE INDEX
import_jobs_org_created_at_idx ON public.import_jobs USING btree (org_id, created_at DESC)
|
| public | import_jobs | import_jobs_org_id_idx | CREATE INDEX
import_jobs_org_id_idx ON public.import_jobs USING btree (org_id)
|
| public | import_jobs | import_jobs_pkey | CREATE UNIQUE
INDEX import_jobs_pkey ON public.import_jobs USING btree (id)
|
| public | import_jobs | import_jobs_profile_id_idx | CREATE INDEX
import_jobs_profile_id_idx ON public.import_jobs USING btree (profile_id)
|
| public | import_jobs | import_jobs_status_idx | CREATE INDEX
import_jobs_status_idx ON public.import_jobs USING btree (status)
|
| public | screening_alerts | screening_alerts_identity_idx | CREATE INDEX
screening_alerts_identity_idx ON public.screening_alerts USING btree (identity_key)
|
| public | screening_alerts | screening_alerts_org_idx | CREATE INDEX
screening_alerts_org_idx ON public.screening_alerts USING btree (org_id)
|
| public | screening_alerts | screening_alerts_org_run_idx | CREATE INDEX
screening_alerts_org_run_idx ON public.screening_alerts USING btree (org_id, run_id)
|

```



```

| public | screening_alerts | screening_alerts_pkey | CREATE UNIQUE
INDEX screening_alerts_pkey ON public.screening_alerts USING btree (id)
|
| public | screening_alerts | screening_alerts_run_identity_idx | CREATE
INDEX screening_alerts_run_identity_idx ON public.screening_alerts USING btree (run_id,
identity_key)
|
| public | screening_alerts | screening_alerts_run_idx | CREATE INDEX
screening_alerts_run_idx ON public.screening_alerts USING btree (run_id)
|
| public | screening_alerts | screening_alerts_run_row_idx | CREATE INDEX
screening_alerts_run_row_idx ON public.screening_alerts USING btree (run_id, row_number)
|
| public | screening_results | screening_results_pkey | CREATE UNIQUE
INDEX screening_results_pkey ON public.screening_results USING btree (id)
|
| public | screening_results | screening_results_run_band_idx | CREATE
INDEX screening_results_run_band_idx ON public.screening_results USING btree (run_id,
risk_band)
|
| public | screening_results | screening_results_run_id_idx | CREATE INDEX
screening_results_run_id_idx ON public.screening_results USING btree (run_id)
|
| public | screening_results | screening_results_run_row_idx | CREATE
INDEX screening_results_run_row_idx ON public.screening_results USING btree (run_id,
row_number)
|
| public | screening_results | screening_results_run_row_ux | CREATE
UNIQUE INDEX screening_results_run_row_ux ON public.screening_results USING btree
(run_id, row_number)
|
| public | screening_runs | screening_runs_import_job_id_idx | CREATE
INDEX screening_runs_import_job_id_idx ON public.screening_runs USING btree
(import_job_id)
|
| public | screening_runs | screening_runs_org_created_at_idx | CREATE
INDEX screening_runs_org_created_at_idx ON public.screening_runs USING btree (org_id,
created_at DESC)
|
| public | screening_runs | screening_runs_org_id_idx | CREATE INDEX
screening_runs_org_id_idx ON public.screening_runs USING btree (org_id)
|
| public | screening_runs | screening_runs_pkey | CREATE UNIQUE
INDEX screening_runs_pkey ON public.screening_runs USING btree (id)
|

```

Perfecto. Entonces no pierdas tiempo afinando histórico de pruebas. Haz una asignación práctica y seguimos.

Como te da igual en entorno de pruebas, te dejo un bloque simple para repartir esos dos lotes de la org de demo@debacu.com, uno a cada hotel.

Voy a usar esta asignación:

- 19566822-c43c-4414-ba4d-b93b4994ec03 → DEMOHOTEL → 26bff2d1-2072-480f-a7c3-e90ea4373a8e

- e9f2a327-03a0-409e-86df-4320ca2d53fd → HOTEL_4 → 146a504b-e6fe-4c4f-8a84-40b24ff179c7

SQL para repartirlos y propagar property_id

```

with mapping as (
  select *
  from (values
    ('19566822-c43c-4414-ba4d-b93b4994ec03'::uuid, '62ec941a-bf2d-4bd3-8dd3-8bf1d95305a3'::uuid, '26bff2d1-2072-480f-a7c3-e90ea4373a8e'::uuid),
    ('e9f2a327-03a0-409e-86df-4320ca2d53fd'::uuid, 'c1d9b932-4fb5-4acb-a081-fe1c160a526d'::uuid, '146a504b-e6fe-4c4f-8a84-40b24ff179c7'::uuid)
  ) as v(import_batch_id, run_id, property_id)
)
update debacu_eval_import_batches b
set property_id = m.property_id
from mapping m
where b.id = m.import_batch_id;
with mapping as (
  select *
  from (values
    ('19566822-c43c-4414-ba4d-b93b4994ec03'::uuid, '62ec941a-bf2d-4bd3-8dd3-8bf1d95305a3'::uuid, '26bff2d1-2072-480f-a7c3-e90ea4373a8e'::uuid),
    ('e9f2a327-03a0-409e-86df-4320ca2d53fd'::uuid, 'c1d9b932-4fb5-4acb-a081-fe1c160a526d'::uuid, '146a504b-e6fe-4c4f-8a84-40b24ff179c7'::uuid)
  ) as v(import_batch_id, run_id, property_id)
)
update import_jobs j
set property_id = m.property_id
from mapping m
where
  nullif(j.summary ->> 'import_batch_id', '')::uuid = m.import_batch_id
  or nullif(j.summary ->> 'run_id', '')::uuid = m.run_id;
with mapping as (
  select *
  from (values
    ('19566822-c43c-4414-ba4d-b93b4994ec03'::uuid, '62ec941a-bf2d-4bd3-8dd3-8bf1d95305a3'::uuid, '26bff2d1-2072-480f-a7c3-e90ea4373a8e'::uuid),
    ('e9f2a327-03a0-409e-86df-4320ca2d53fd'::uuid, 'c1d9b932-4fb5-4acb-a081-fe1c160a526d'::uuid, '146a504b-e6fe-4c4f-8a84-40b24ff179c7'::uuid)
  ) as v(import_batch_id, run_id, property_id)
)
update screening_runs r
set property_id = m.property_id
from mapping m
where r.id = m.run_id;
update screening_results sr
set property_id = r.property_id
from screening_runs r
where sr.run_id = r.id
  and r.property_id is not null;
update screening_alerts sa
set property_id = r.property_id
from screening_runs r
where sa.run_id = r.id
  and r.property_id is not null;
update debacu_eval_guest_stays s
set property_id = b.property_id
from debacu_eval_import_batches b
where s.import_batch_id = b.id
  and b.property_id is not null;

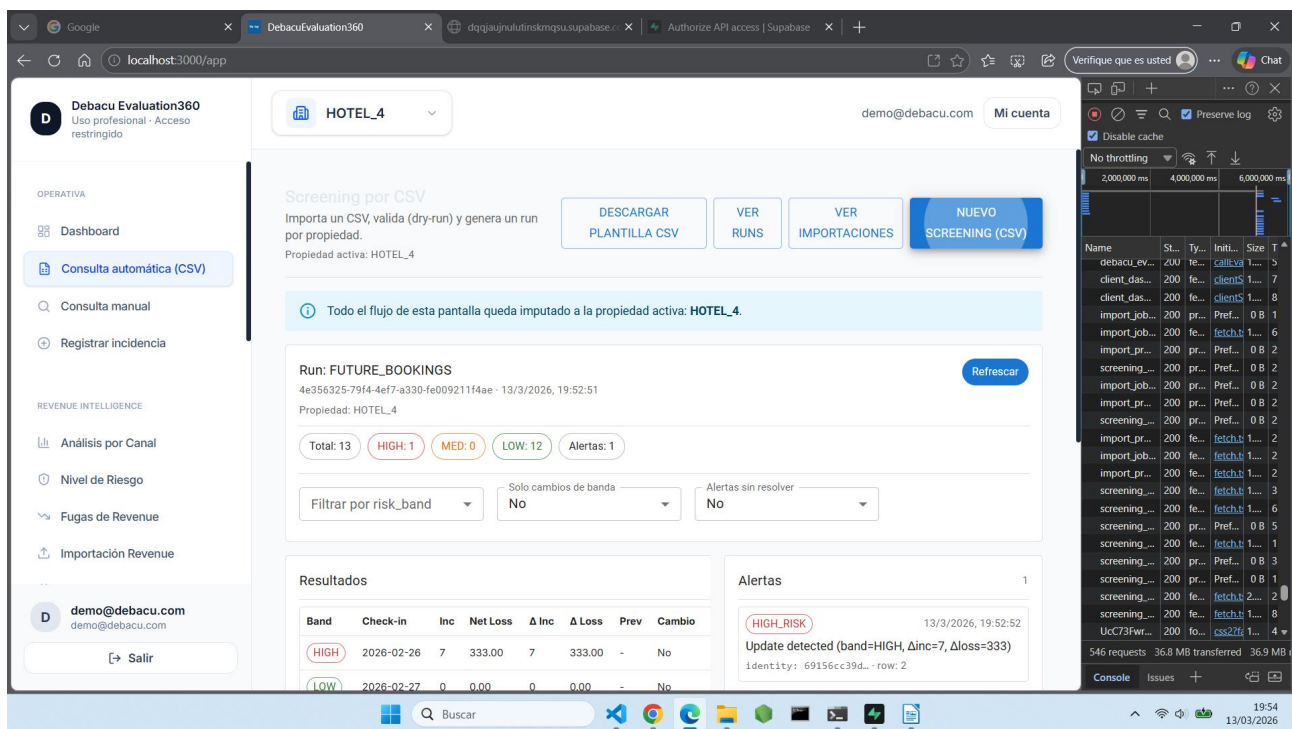
```

Verificación rápida

```
select id, org_id, property_id, file_name, created_at
from debacu_eval_import_batches
where org_id = 'b5909245-9bb5-488e-a7f2-4fb4762922a1'
order by created_at desc;
select id, org_id, property_id, run_type, created_at
from screening_runs
where org_id = 'b5909245-9bb5-488e-a7f2-4fb4762922a1'
order by created_at desc;
select count(*) as stays_without_property
from debacu_eval_guest_stays
where org_id = 'b5909245-9bb5-488e-a7f2-4fb4762922a1'
and property_id is null;
```

Cuando eso esté, el siguiente paso bueno ya es **dejar de parchear histórico** y cambiar la Edge Function para que todo nuevo nazca ya con `property_id`.

PAZNTALLA FINALIZADA



ATENCIÓN PLAN BÁSICO CONSULTA MANUAL HAY QUE HABILITAR PLAN BÁSICO PARA PODER USAR CSV

Bien. Entonces esta pantalla ya queda encarrilada.

El siguiente paso lógico no es retocar más esta vista, sino cerrar el flujo completo de screening con estas dos piezas:

1. Consulta manual

- añadir selector de propiedad en el formulario
- guardar la incidencia/consulta contra esa propiedad

- dejar claro si la consulta afecta a org o a propiedad

2. **Registrar incidencia**

- selector de propiedad obligatorio
- persistir `property_id`
- luego eso te permitirá explotar riesgo por hotel de verdad

CONTINUAMOS CON CONSULTA MANUAL (TAMBIEN CON SELECTOR DE PROPIEDAD PARA ESTADISTICAS POT HOTEL)